



Introduzione alle applicazioni Web

Prof. Michele Ciavotta
Corso di Sistemi Distribuiti

Outline

- Introduzione alle Applicazioni Web
 - Il Web come piattaforma
 - Architettura di un'applicazione Web
- **Client side:** HTML
- **Server side:** Servlet
 - Il modello
 - HTTP Servlet
- **Server side:** JSP
 - Il modello
 - Struttura di una JSP
 - Gli elementi che compongono una JSP
 - JSP e JavaBeans
- Il pattern MVC



Il Web statico

- Alle origini del Web: il server restituisce **documenti HTML statici** memorizzati su disco
- Il client richiede una risorsa tramite **URL**, il server la individua e la invia “così com’è”
- Il contenuto è **uguale per tutti gli utenti** e non dipende da parametri o interazioni
- Nessuna elaborazione lato server: si comporta come un **file system remoto** esposto via HTTP
- I **form HTML** introducono (html 2.0 – 1995) il primo meccanismo di invio dati dal client al server → inizio del **web dinamico**
- Servono risposte personalizzate, gestione di dati utente, accesso a database → nascono le **Web Application**

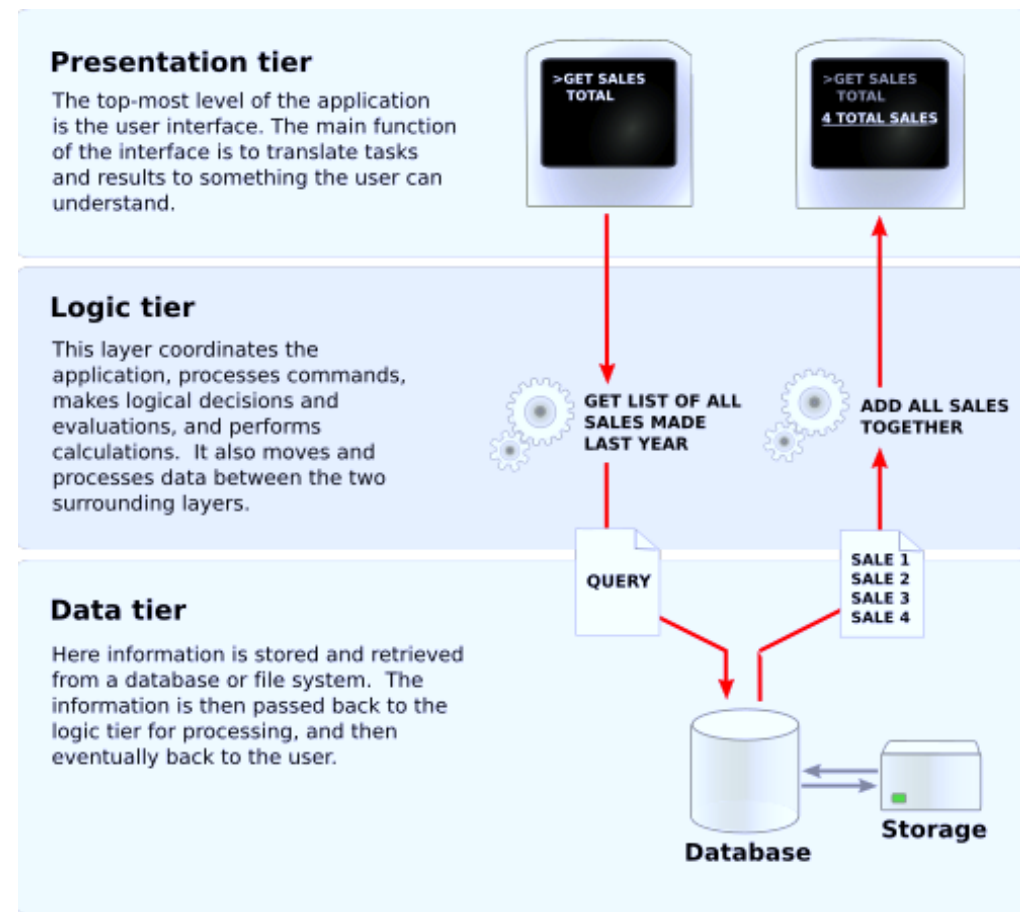


Una **applicazione web**, abbreviato **web app**, [...] indica genericamente tutte le **applicazioni distribuite** ovvero applicazioni accessibili/fruibili via web per mezzo di una rete, [...] o attraverso **internet**, cioè in una architettura tipica di tipo client-server, offrendo determinati servizi all'utente client.

https://it.wikipedia.org/wiki/Applicazione_web

APPLICAZIONI WEB

- Estende l'idea di pagina web offrendo funzionalità complesse
- Il server non si limita a ricercare e fornire un documento da mandare al client ma implementa funzionalità



Perché il web?

- **Basato su Internet** (non è la stessa cosa!)
 - Ambiente standard (alla base c'è TCP/IP)
 - Ampia diffusione, conoscenza diffusa
- **Semplicità e uniformità** dell'interazione
 - Interfaccia grafica (eseguita da Browsers)
 - Interazione attraverso un protocollo standard (HTTP)
 - Definizione di una API - *Application Programming Interface* (per es. REST, vedremo meglio)
- **Infrastruttura matura ed estesa**
 - Supporta sistemi aperti (connessione dinamica di nuovi componenti)
 - Strumenti sempre più completi e potenti: evoluzioni di HTML, Javascript, WASM ...



Caratteristiche principali – Protocollo di comunicazione

Applicazione Web = adozione del protocollo HTTP

Caratteristiche del protocollo HTTP

- **Formato a caratteri** (lento): occorre tradurre e ritradurre i dati (es. da testo a numeri: da “256” a 256)
- **Header** (per metadati) e **Body** (corpo del messaggio)
- Utilizzo del linguaggio HTML per input e output:
 - Uso di **FORM** per l’acquisizione dati (invio dati al server)
 - Uso di documenti (es. HTML) in risposta (dal server verso il client)
 - Possibilità di pagine dinamiche (*JavaScript* con dati scambiati in JSON)
- Utilizzo di payload di tipo **MIME (Multimedia Internet Mail Extensions)**
 - Per generalizzare oltre HTML (xml, json, zip, jpeg)
- **Conversazioni (interazioni client-server) prive di stato (memoria)**
 - Ogni richiesta è un messaggio autonomo, indipendente dagli altri
 - Per creare sessioni di lavoro (legare più richieste tra loro) servono informazioni esplicite
 - Cookie
 - Campi nascosti (aggiunti alla richiesta)

Un **cookie HTTP** (web cookie, browser cookie) è un piccolo blocco di dati che un server invia al browser web di un utente. Il browser può memorizzare il cookie e rinviarlo allo stesso server con richieste successive. In genere, un cookie viene utilizzato per stabilire se due richieste provengono dallo stesso browser, ad esempio per mantenere un utente loggato.



Dal Web alle Applicazioni Web

Il Web supporta l'interazione tra client e server (soprattutto via HTTP)

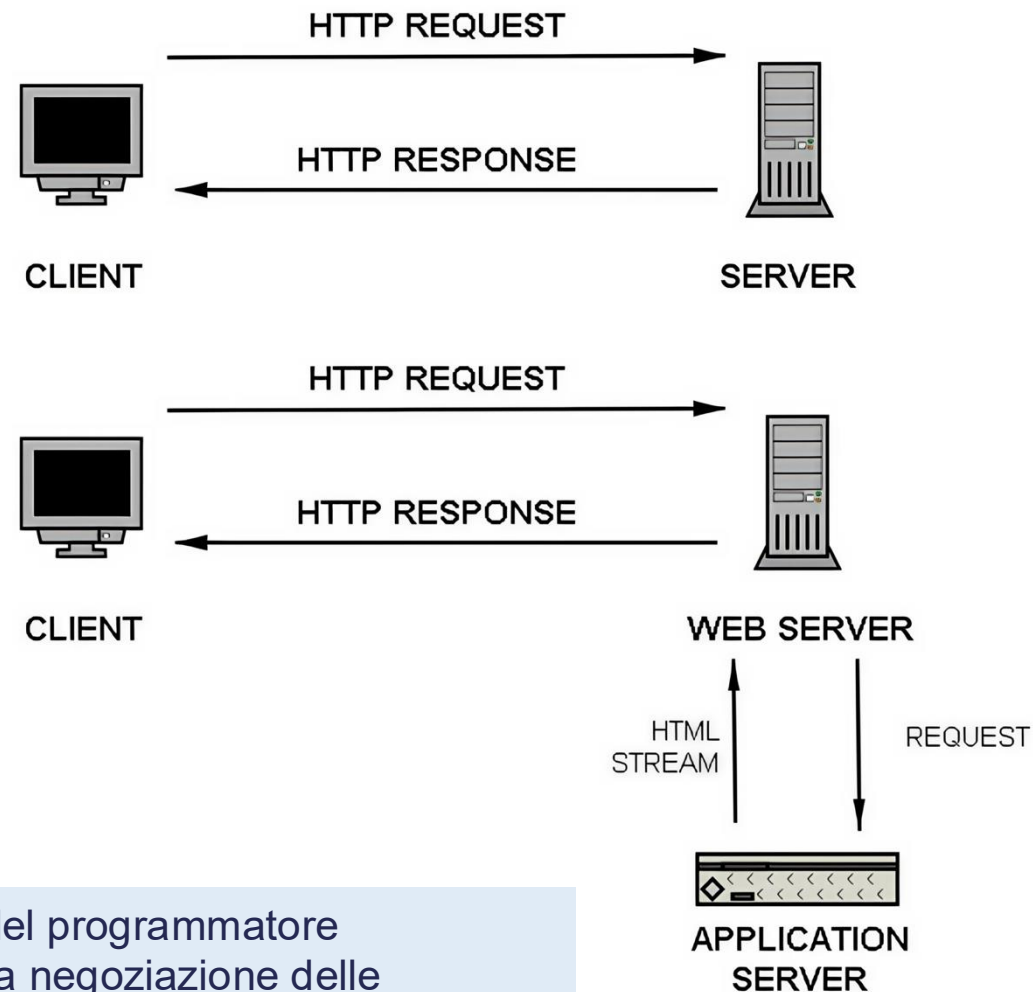
Ben presto il web si è evoluto andando al di là del trasferimento di pagine web (file HTML).

È diventato uno strumento per fornire accesso ad applicazioni remote.

Per eseguire una applicazione, il Web Server utilizza un **Application Server**

- Un Application server si caratterizza dal *protocollo di interazione con il Web Server*
- L'interazione con il client avviene sempre tramite il web server

Questo modello è importante perché definisce il confine di responsabilità del programmatore applicativo, che non deve preoccuparsi della gestione del socket TCP, della negoziazione delle connessioni o della concorrenza tra client diversi: queste responsabilità sono delegate all'infrastruttura



Tecnologia Server Side

La computazione avviene (in parte) lato server e può avvenire tramite l'esecuzione **programmi (compilati)** o **script (interpretati)**

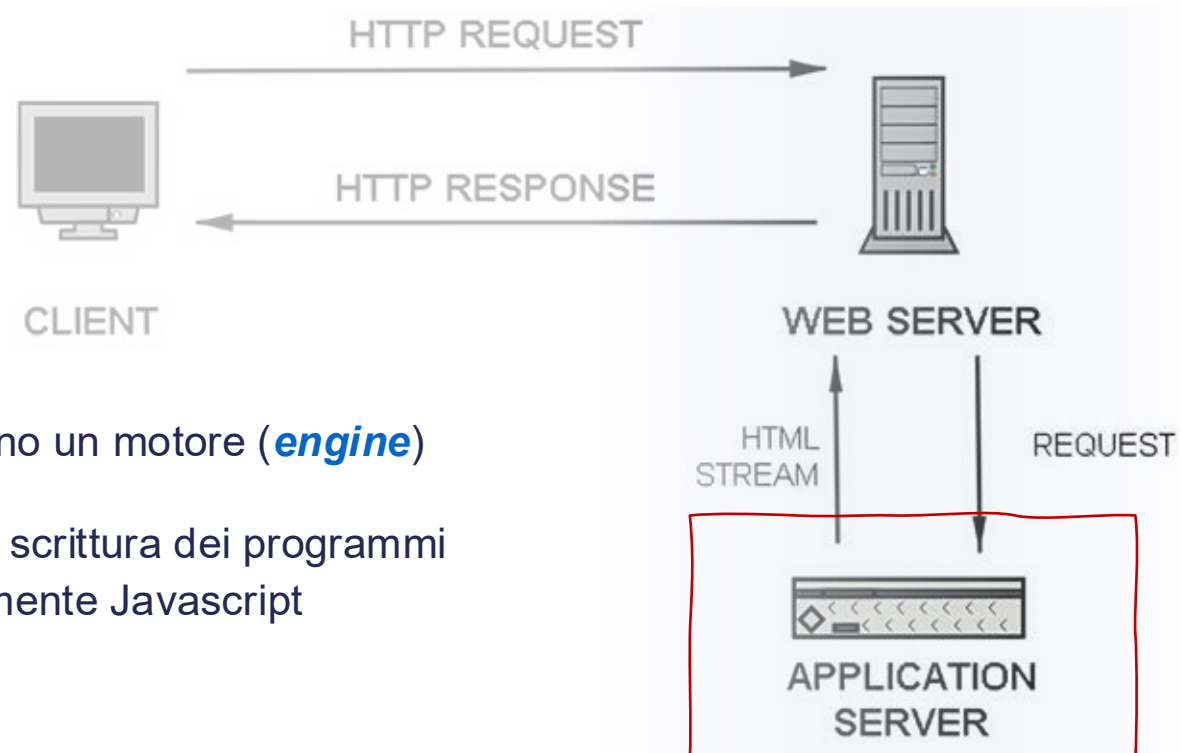
- L'application server fornisce un ambiente per la gestione automatizzata del **ciclo di vita** di tali programmi

Nel caso di **programmi compilati** l'application server si limita ad invocare, su richiesta del client, un eseguibile

- L'eseguibile può essere scritto in un qualsiasi linguaggio che supporti l'interazione con il web server
- Tra i linguaggi più diffusi: C e C++

Nel caso di esecuzione di **script**, l'application server ha al suo interno un motore (**engine**) in grado di interpretare il linguaggio di scripting usato

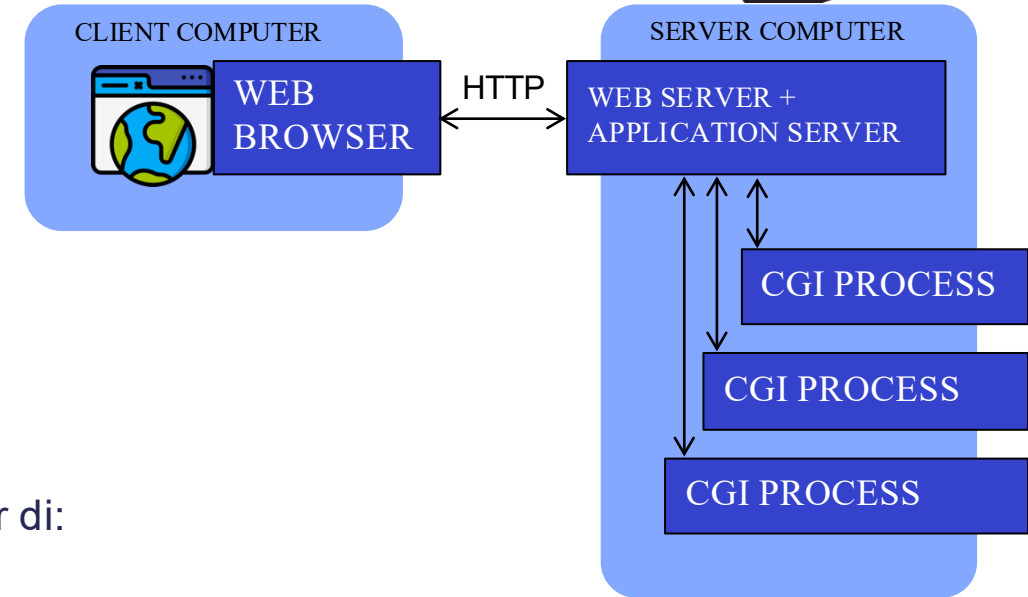
- Si perde in velocità di esecuzione, ma si guadagna in facilità di scrittura dei programmi
- Tra i linguaggi più diffusi Java, PHP, Python e Perl, più recentemente Javascript (Nodejs)



Architettura di un'applicazione web compilata

- **Uniform Resource Locator (URL)**
 - definisce un naming globale
 - **HyperText Transfer Protocol (HTTP)**
 - permette di invocare i programmi sul server come fossero risorse
- Validi anche per applicazioni interpretate

- **Common Gateway Interface (CGI)**. Protocollo che permette al server di:
 - attivare un programma (crea un processo)
 - passargli le richieste e i parametri provenienti dal client
 - recuperare la risposta
- Ogni applicazione CGI deve quindi conformarsi alle regole di input/output
 - Leggere l'input (es. *stdin* per dati POST).
 - Leggere le variabili d'ambiente (es. *QUERY_STRING*, *REQUEST_METHOD*).
 - Scrivere l'output su *stdout*, includendo intestazioni HTTP e corpo della risposta.
 - Il resto è gestito dall'application server



Quando un client (come un browser) invia una richiesta HTTP a un server web, specifica il percorso relativo a una risorsa, che fa parte di un URL.

- Se il percorso richiesto corrisponde a un documento HTML statico, il server restituisce il contenuto del file così com'è, senza ulteriori elaborazioni.
- Se invece il percorso corrisponde a un'applicazione CGI, il server esegue il programma associato e genera dinamicamente la risposta da inviare al client.

Architettura di un'applicazione web interpretata

Il server delega all'interprete del linguaggio (o a un middleware) la gestione del protocollo CGI.

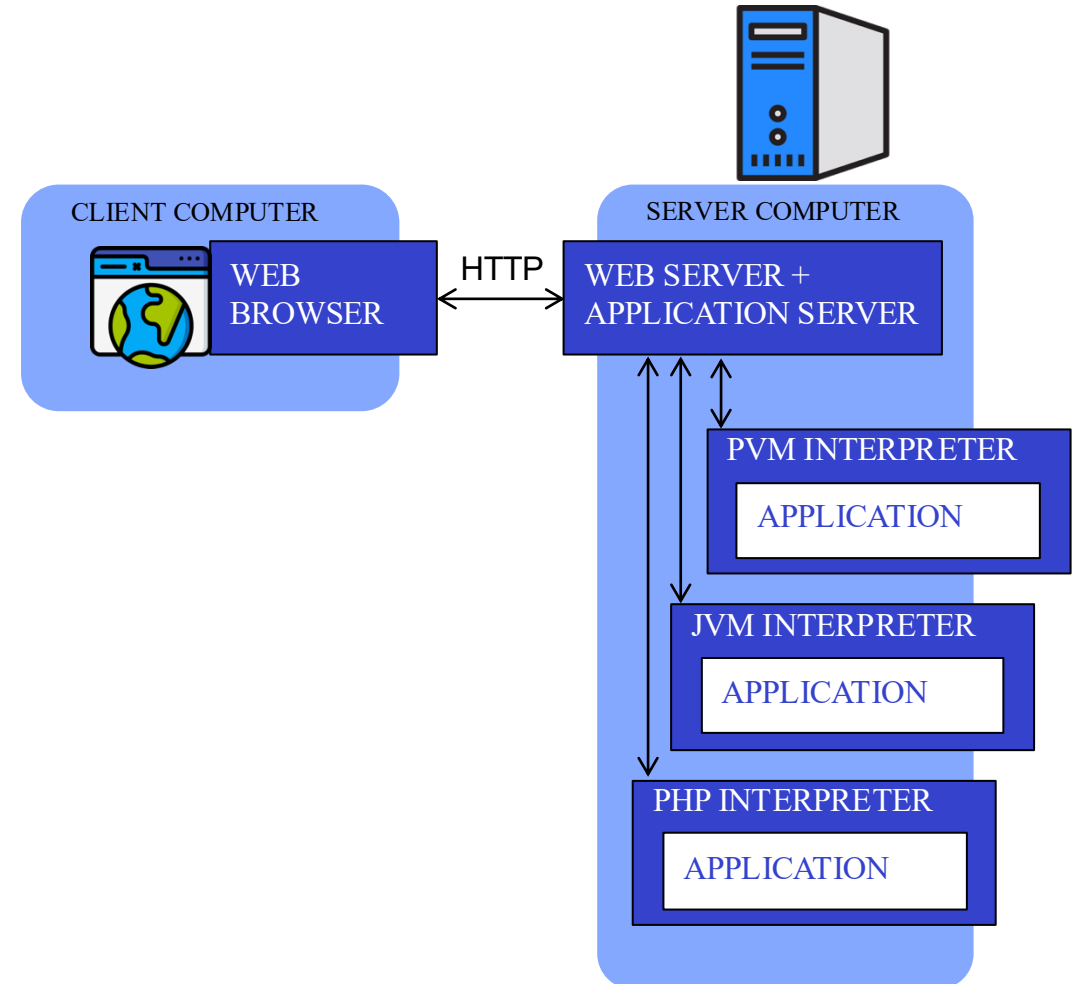
- Le applicazioni non interagiscono direttamente con il protocollo, ma si limitano a definire la logica applicativa, secondo le convenzioni del linguaggio usato.

Vantaggi

- Separazione delle responsabilità:** lo sviluppatore si concentra solo sulla logica dell'applicazione.
- Aderenza al modello del linguaggio:** l'applicazione segue un paradigma coerente con l'ecosistema di sviluppo.
- Maggiore semplicità, portabilità e manutenibilità.**

Esempi

- Python:** esecuzione tramite moduli WSGI o framework (Flask, Django)
- Java:** Servlet gestite da container (Tomcat, Jetty)
- PHP:** integrato con moduli del server web (mod_php, FPM)
- Node.js:** runtime JavaScript



CGI come nucleo concettuale dell'applicazione web

Abbiamo visto due architetture — compilata e interpretata. Fermiamoci: cosa hanno in comune?

Dietro ogni servlet, framework WSGI o handler Node c'è la stessa idea: un pezzo di codice che riceve una **request** e produce una **response**. Quella è l'eredità del CGI.

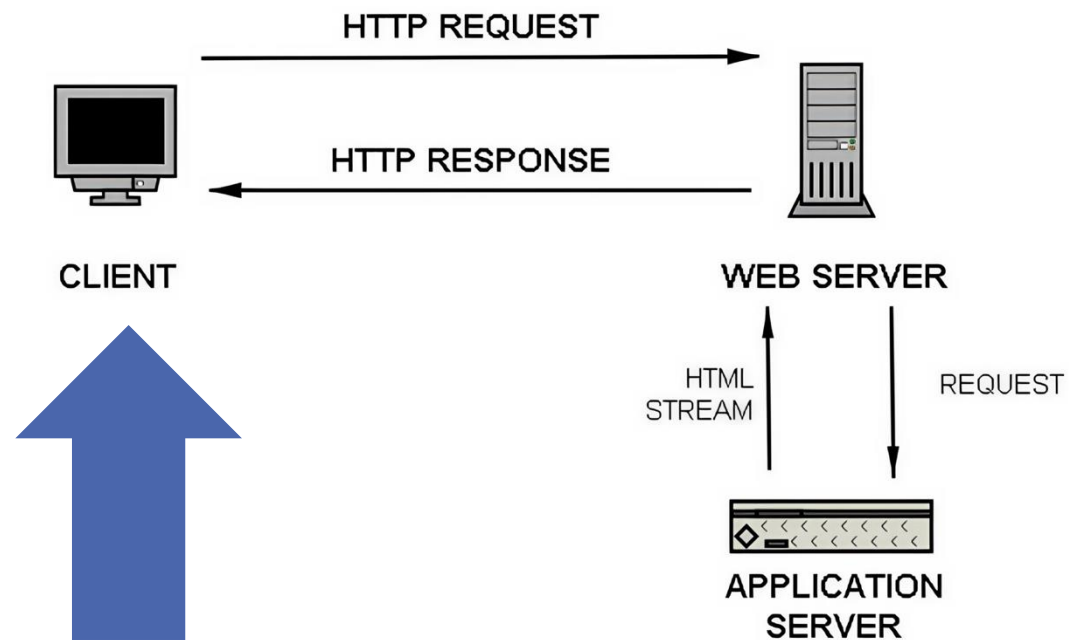
Cosa resta del CGI

- **Il modello:** request → codice applicativo → response
- **Il contratto I/O:** input strutturato (parametri, headers, body), output strutturato (headers + body)
- **La risorsa attivabile:** l'URL mappa un pezzo di logica, non un file

Cosa è stato superato

- **Process-per-request:** un fork per richiesta non scala → servlet, FPM, runtime persistenti
- **Stato nel processo:** il processo muore a ogni request → niente pool di connessioni, sessioni solo tramite storage esterno (cookie + file/DB) → container con memoria persistente
- **I/O a basso livello:** stdin/stdout e variabili d'ambiente → API del linguaggio (oggetti req/res)

*Il CGI «vero» è storia, ma il core di ogni web app — il codice che trasforma una request HTTP in una response — è ancora concettualmente uno script CGI. MVC, servlet e framework sono l'infrastruttura costruita **attorno** a questo nucleo.*



Anatomia di un'applicazione web

CLIENT SIDE - HTML



HTML (HyperText Markup Language)

HTML è il linguaggio standard per la strutturazione dei contenuti delle pagine web.

- Non è un linguaggio di programmazione, ma un **linguaggio di marcatura** che descrive la **struttura** e la semantica delle informazioni visualizzate in un browser.

Ruolo nel modello web

- L'HTML rappresenta la parte statica dell'interfaccia utente.
- Viene inviato dal server e interpretato dal browser.
- È spesso affiancato da **CSS** (per la presentazione) e **JavaScript** (per l'interattività).

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <title>Pagina HTML Minimale</title>
</head>
<body>
  <h1>La Mia Prima Pagina HTML</h1>
  <p>Questo è un esempio minimale di una pagina HTML in italiano.</p>
  <ul>
    <li>Elemento 1</li>
    <li>Elemento 2</li>
    <li>Elemento 3</li>
  </ul>
  <p>Clicca <a href="#">qui</a> per un link di esempio.</p>
</body>
</html>
```

Interazioni con il server basate su link

- Un link in un documento HTML può essere usato per puntare ad una risorsa remota

```
<p>Click the link to ask the servlet to send back an HTML document</p>
```

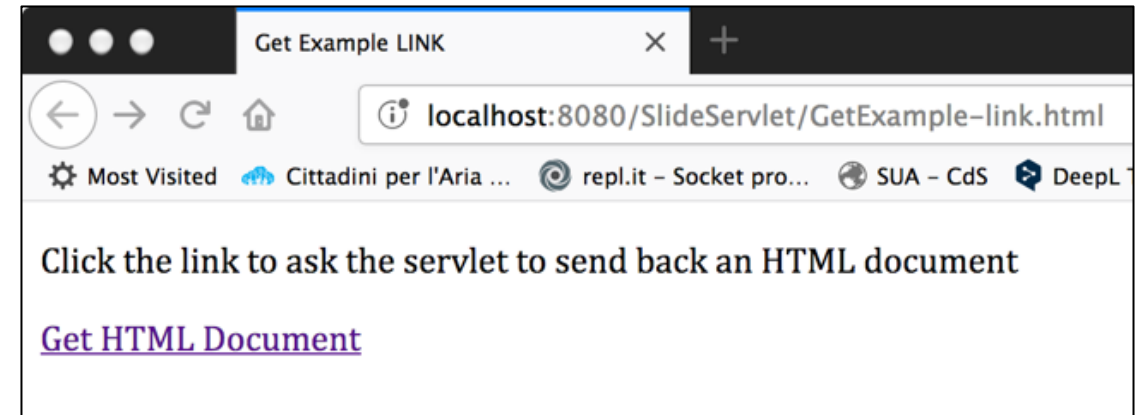
```
<a href="http://localhost:8080/SlideServlet/GetHTTPServlet">
```

```
    Get HTML Document
```

```
</a>
```

- Il browser invia richieste del tipo:

```
GET /SlideServlet/GetHTTPServlet HTTP/1.1
```



Interazioni con il server per mezzo di Form

- Anche il parametro `action` di un *Form* può essere usato per puntare ad una risorsa remota (file o applicazione)

```
<form action="http://localhost:8080/SlideServlet/GetHTTPServlet" method="GET">
```

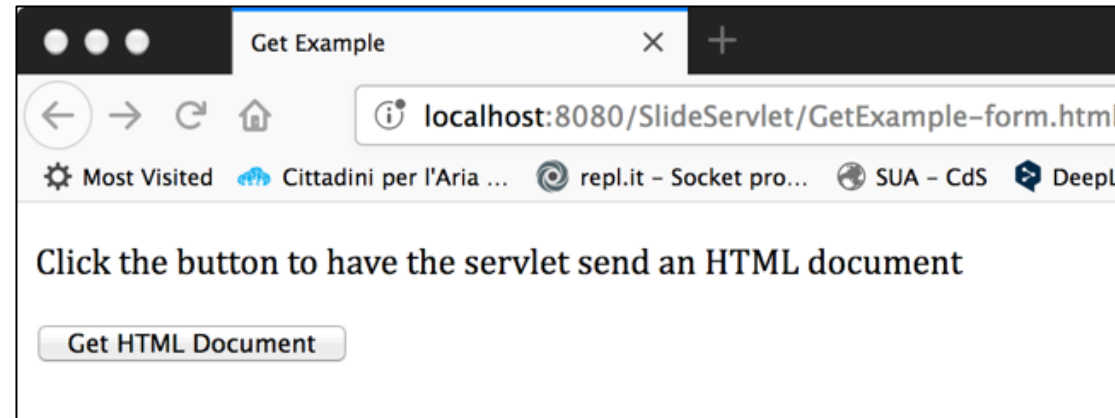
```
<p>Click the button to have the servlet send an HTML document</p>
```

```
<input type="submit" value="Get HTML Document">
```

```
</form>
```

- Il browser invia una richiesta del tipo:

```
GET /SlideServlet/GetHTTPServlet HTTP/1.1
```



Interazioni con il server per mezzo di Form

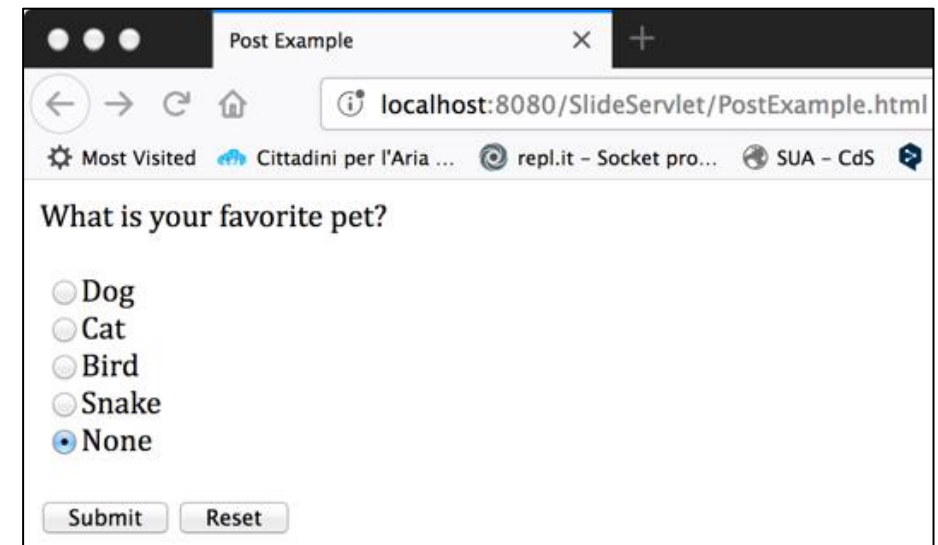
- Un FORM può essere usato per mandare dati al server (chiedendoli all'utente)

```
<form action="http://localhost:8080/SlideServlet/PostHTTPServlet" method="POST">
  What is your favorite pet?<br> <br>
  <input type="radio" name="animal" value="dog">Dog<br>
  <input type="radio" name="animal" value="cat">Cat<br>
  <input type="radio" name="animal" value="bird">Bird<br>
  <input type="radio" name="animal" value="snake">Snake<br>
  <input type="radio" name="animal" value="none" checked>None<br>
  <br> <input type="submit" value="Submit"> <input type="reset">
</form>
```

- Il browser invia una richiesta del tipo:

```
POST /SlideServlet/PostHTTPServlet HTTP/1.1
Content-Length: 11
Content-Type: application/x-www-form-urlencoded

animal=none
```



Post Example

localhost:8080/SlideServlet/PostExample.html

What is your favorite pet?

☐ Dog

☐ Cat

☐ Bird

☐ Snake

☒ None

Submit Reset

Interazioni con il server per mezzo di Form

- Un FORM può essere usato per mandare dati al server (chiedendoli all'utente)

```
<form action="http://localhost:8080/SlideServlet/PostHTTPServlet" method="POST">
  What is your favorite pet?<br> <br>
  <input type="radio" name="animal" value="dog">Dog<br>
  <input type="radio" name="animal" value="cat">Cat<br>
  <input type="radio" name="animal" value="bird">Bird<br>
  <input type="radio" name="animal" value="snake">Snake<br>
  <input type="radio" name="animal" value="none" checked>None<br>
  <br> <input type="submit" value="Submit"> <input type="reset">
</form>
```

- Il browser invia una richiesta del tipo:

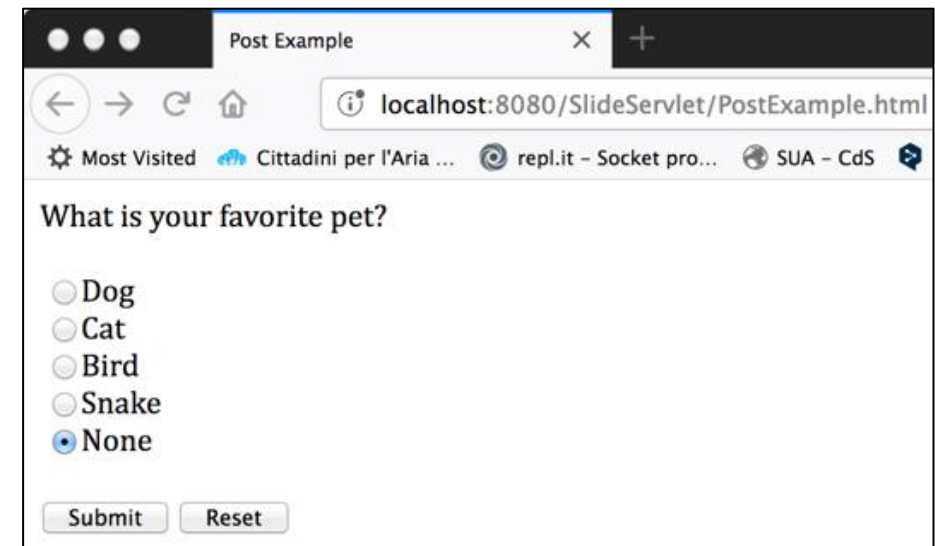
```
POST /SlideServlet/PostHTTPServlet HTTP/1.1
```

```
Content-Length: 11
```

```
Content-Type: application/x-www-form-urlencoded
```

```
animal=none
```

Headers



Post Example

localhost:8080/SlideServlet/PostExample.html

What is your favorite pet?

☐ Dog

☐ Cat

☐ Bird

☐ Snake

☒ None

Submit Reset

Interazioni con il server per mezzo di Form

- Un FORM può essere usato per mandare dati al server (chiedendoli all'utente)

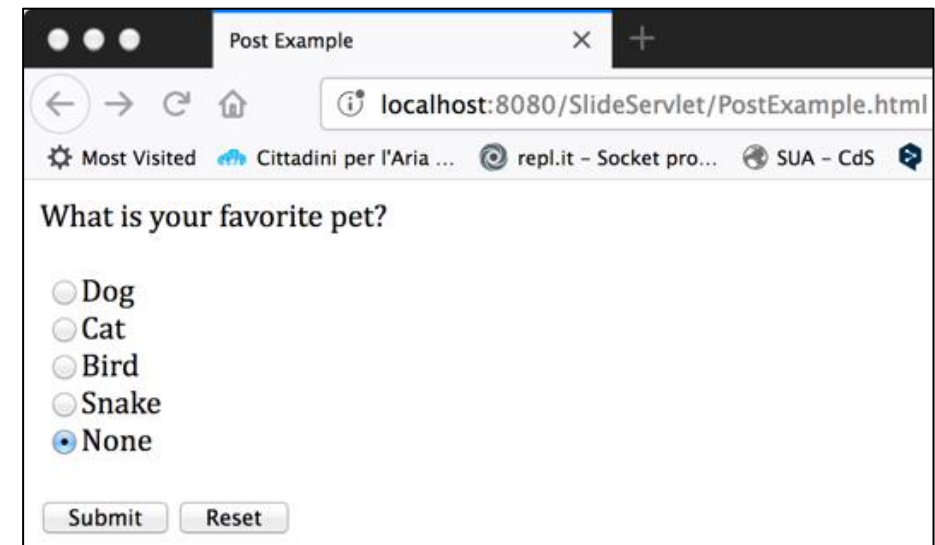
```
<form action="http://localhost:8080/SlideServlet/PostHTTPServlet" method="POST">
  What is your favorite pet?<br> <br>
  <input type="radio" name="animal" value="dog">Dog<br>
  <input type="radio" name="animal" value="cat">Cat<br>
  <input type="radio" name="animal" value="bird">Bird<br>
  <input type="radio" name="animal" value="snake">Snake<br>
  <input type="radio" name="animal" value="none" checked>None<br>
  <br> <input type="submit" value="Submit"> <input type="reset">
</form>
```

- Il browser invia una richiesta del tipo:

```
POST /SlideServlet/PostHTTPServlet HTTP/1.1
Content-Length: 11
Content-Type: application/x-www-form-urlencoded
```

```
animal=none
```

Payload



Post Example

localhost:8080/SlideServlet/PostExample.html

What is your favorite pet?

☐ Dog

☐ Cat

☐ Bird

☐ Snake

☒ None

Submit Reset

Thank you!

localhost:8080/SlideServlet/PostHTTPServlet

Thank you for participating.
Results:

dog: 0.00% responses: 0

cat: 66.67% responses: 4

bird: 16.67% responses: 1

snake: 16.67% responses: 1

none: 0.00% responses: 0

Total responses: 6

Elements Console Sources **Network** Performance Memory Application Security

Filter ☐ Invert ☐ Hide data URLs **All** Fetch/XHR JS CSS Img Media Font Doc WS Wasm Manifest Other

☐ Has blocked cookies ☐ Blocked Requests ☐ 3rd-party requests

☒ Use large request rows ☐ Group by frame

☐ Show overview ☐ Capture screenshots

Name **PostHTTPServlet /SlideServlet**

General

Request URL: http://localhost:8080/SlideServlet/PostHTTPServlet

Request Method: POST

Status Code: 200

Remote Address: [::1]:8080

Referrer Policy: strict-origin-when-cross-origin

Response Headers View source

Content-Length: 271

Content-Type: text/html; charset=ISO-8859-1

Date: Thu, 24 Mar 2022 12:49:25 GMT

Request Headers View source

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9

Accept-Encoding: gzip, deflate, br

Accept-Language: en-GB,en-US;q=0.9,en;q=0.8

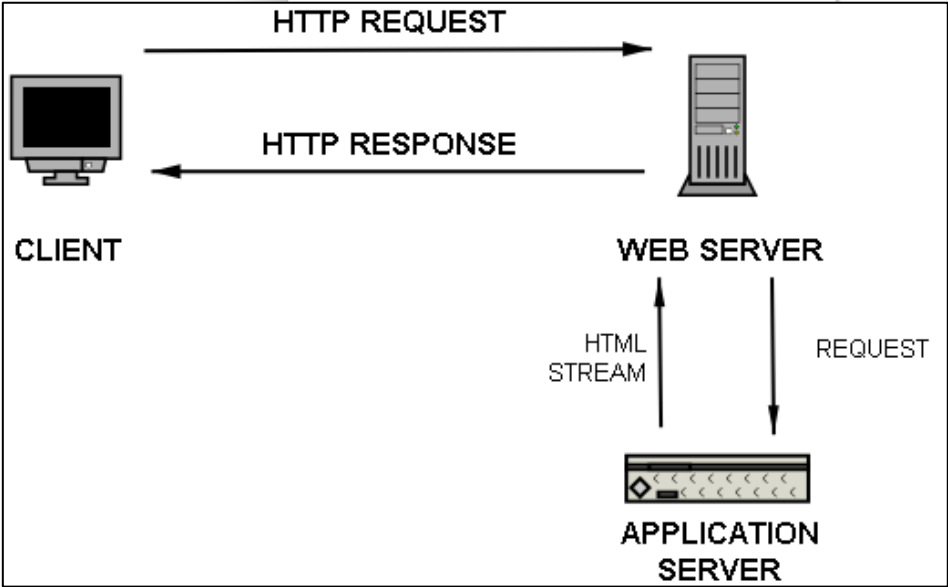
Cache-Control: max-age=0

Connection: keep-alive

Content-Length: 12

Content-Type: application/x-www-form-urlencoded

POST HTTP
request/
response
with data



POST HTTP request/response with data

The screenshot shows a web browser window with the address bar displaying `localhost:8080/SlideServlet/PostHTTPServlet`. The page content says "Thank you for participating. Results:" followed by a list of animal categories and their response counts:

```
dog: 0.00%  responses: 0
cat: 66.67%  responses: 4
bird: 16.67%  responses: 1
snake: 16.67%  responses: 1
none: 0.00%  responses: 0
```

At the bottom, it states "Total responses: 6". The browser's developer tools are open to the Network tab, showing a single request named "PostHTTPServlet /SlideServlet". The request's payload is visible as "animal=sake".

Form e dati strutturati: JSON e XML

Domanda: una form può mandare dati in JSON o XML?

Submit nativo: solo 3 enctype

- `application/x-www-form-urlencoded` (default)
- `multipart/form-data` (upload file)
- `text/plain` (raro)

→ **JSON e XML non esistono come enctype.**

Richiesta HTTP risultante

```
POST /PostHTTPServlet HTTP/1.1
Content-Type: application/json
Content-Length: 20

{"animal": "dog"}
```

Soluzione: intercettare il submit con JS

```
<form id="petForm">
  ... campi identici alla slide precedente ...
</form>

<script>
const f = document.getElementById("petForm");
f.addEventListener("submit", async e => {
  e.preventDefault();
  const data = { animal: f.animal.value };
  await fetch("/PostHTTPServlet", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });
});
</script>
```

La form HTML resta come UI per raccogliere i dati; fetch costruisce la richiesta HTTP con il Content-Type voluto. Per il server cambia solo il parsing del body.

Interazioni con il server per mezzo di Form

- I dati possono essere inviati con una Form anche usando il metodo GET

```
<form action="http://localhost:8080/SlideServlet/GetPostHTTPServlet" method="GET">  
  What is your favorite pet?<br> <br>  
  <input type="radio" name="animal" value="dog">Dog<br>  
  <input type="radio" name="animal" value="cat">Cat<br>  
  <input type="radio" name="animal" value="bird">Bird<br>  
  <input type="radio" name="animal" value="snake">Snake<br>  
  <input type="radio" name="animal" value="none" checked>None<br>  
  <br> <input type="submit" value="Submit"> <input type="reset">  
</form>
```

- Il browser invia una richiesta del tipo:

```
GET /SlideServlet/GetPostHTTPServlet?animal=none HTTP/1.1
```



What is your favorite pet?

☐ Dog

☐ Cat

☐ Bird

☐ Snake

☒ None

Submit Reset

Interazioni con il server per mezzo di Form

- I dati possono essere inviati con un FORM usando il metodo GET

```
<form action="http://localhost:8080/SlideServlet/GetPostHTTPServlet" method="GET">  
  What is your favorite pet?<br> <br>  
  <input type="radio" name="animal" value="dog">Dog<br>  
  <input type="radio" name="animal" value="cat">Cat<br>  
  <input type="radio" name="animal" value="bird">Bird<br>  
  <input type="radio" name="animal" value="snake">Snake<br>  
  <input type="radio" name="animal" value="none" checked>None<br>  
  <br> <input type="submit" value="Submit"> <input type="reset">  
</form>
```

- Il browser invia una richiesta del tipo:

GET /SlideServlet/GetPostHTTPServlet?animal=none HTTP/1.1

Query parameter

What is your favorite pet?

☐ Dog

☐ Cat

☐ Bird

☐ Snake

☒ None

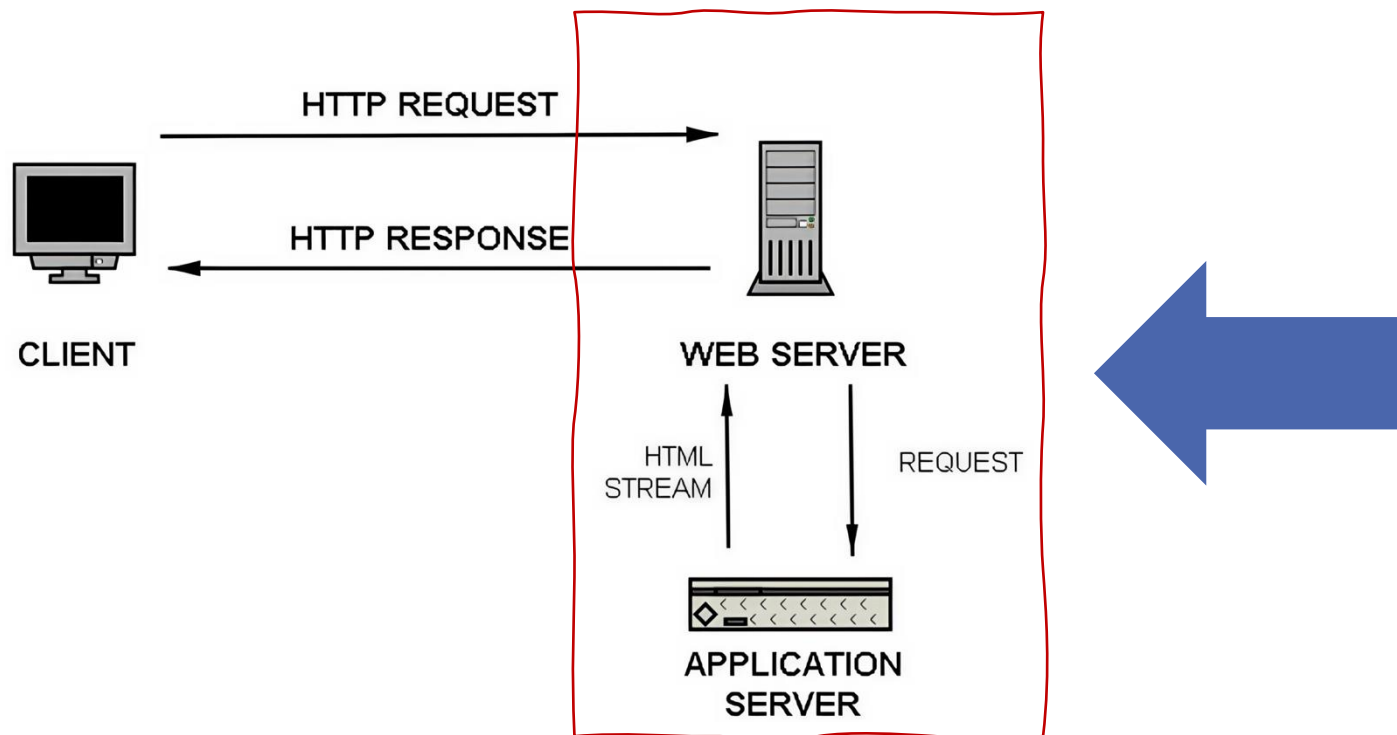
Submit Reset

GET HTTP request/response with data

The screenshot shows a web browser window with the address bar displaying `localhost:8080/SlideServlet/GetPostHTTPServlet?animal=bird`. The page content shows "Thank you!" and "Results: bird: 28.57% responses: 2". The Network tab is open, showing a list of requests. The selected request is `GetPostHTTPServlet?ani.../SlideServlet`. The request details are as follows:

- Request URL:** `http://localhost:8080/SlideServlet/GetPostHTTPServlet?animal=bird` (highlighted with a red box)
- Request Method:** GET
- Status Code:** 200
- Remote Address:** `:::1]:8080`
- Referrer Policy:** `strict-origin-when-cross-origin`
- Response Headers:**
 - Content-Length:** 147
 - Content-Type:** `text/html; charset=ISO-8859-1`
 - Date:** `Fri, 25 Mar 2022 09:43:35 GMT`
- Request Headers:**
 - Accept:** `text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9`
 - Accept-Encoding:** `gzip, deflate, br`
 - Accept-Language:** `en-GB,en-US;q=0.9,en;q=0.8`
 - Connection:** `keep-alive`
 - Host:** `localhost:8080`
 - Referer:** `http://localhost:8080/SlideServlet/GetExample-form-data.html`
 - sec-ch-ua:** `" Not A;Brand";v="99", "Chromium";v="99", "Google Chrome";v="99"`

At the bottom of the Network tab, it shows "1 requests" and "266 B transferred".



SERVER SIDE - JAVA SERVLET



Java Servlets

- Sono piccole applicazioni Java contenute ed eseguite dal (application) server
 - Esempio: Apache Tomcat
- Una servlet è un **componente** gestito in modo *automatico* da un **container** (detto anche **engine**)
- Deve implementare un'interfaccia prestabilita che definisce il set di metodi
 - **Vantaggi**: semplicità e standardizzazione
 - **Svantaggi**: rigidità del modello
- Il container controlla le servlet (le attiva/disattiva) in base alle richieste dei client. Questo è possibile in maniera automatica perché *le servlet implementano una interfaccia nota al server*
- Le servlet sono *oggetti Java residenti nella memoria del server*
 - il codice dei metodi delle servlet viene eseguito da *thread creati e gestiti dall'application server*
 - possono interagire con altre servlet

Sulla mancanza di stato

- HTTP non prevede persistenza dello stato della conversazione tra una interazione e la successiva (*stateless*)
 - Non si possono mantenere informazioni tra un messaggio e i successivi
 - Non si possono identificare i client
- Mantenere lo stato della conversazione è compito dell'applicazione (se vuole)
 - **Cookies**
 - Informazioni memorizzate a livello di client
 - Permettono di gestire sessioni di lavoro
 - **HTTPSession**
 - Oggetto gestito automaticamente dal container/engine (con cookie o riscrittura delle URL)
 - Le servlet possono accedervi per immagazzinare informazioni

Interfaccia Servlet

- Ogni servlet implementa l'interfaccia `jakarta.servlet.Servlet`, con 5 metodi

`void init(ServletConfig config)` - Inizializza la servlet, viene invocato dopo la creazione della stessa

`void destroy()` - Chiamata quando la servlet termina (es: per chiudere un file o una connessione con un database)

`void service(ServletRequest request, ServletResponse response)` - Invocato per gestire le richieste dei client

`ServletConfig getServletConfig()` - Restituisce i parametri di inizializzazione e il `ServletContext` che dà accesso all'ambiente

`String getServletInfo()` - Restituisce informazioni tipo autore e versione

Classi astratte

- L'interfaccia è solo la dichiarazione dei metodi che, per essere utilizzabili, devono essere implementati in una classe
- Sono presenti due **classi astratte**, cioè che implementano i metodi dell'interfaccia in modo che non facciano nulla.

`jakarta.servlet.GenericServlet` - Definisce metodi indipendenti dal protocollo

`jakarta.servlet.http.HttpServlet` - Definisce metodi per l'uso in ambiente web

- Questo semplifica l'implementazione delle servlet vere e proprie in quando basta implementare (ridefinendoli) solo i metodi che interessano

La classe `HttpServlet`

- Implementa `service()` in modo da invocare i metodi per servire le richieste dal web (router)

Metodi `doX`

X è un metodo HTTP (`doGet`, `doPost`, ...)

`doX` è dedicato alle richieste di tipo X

Parametri:

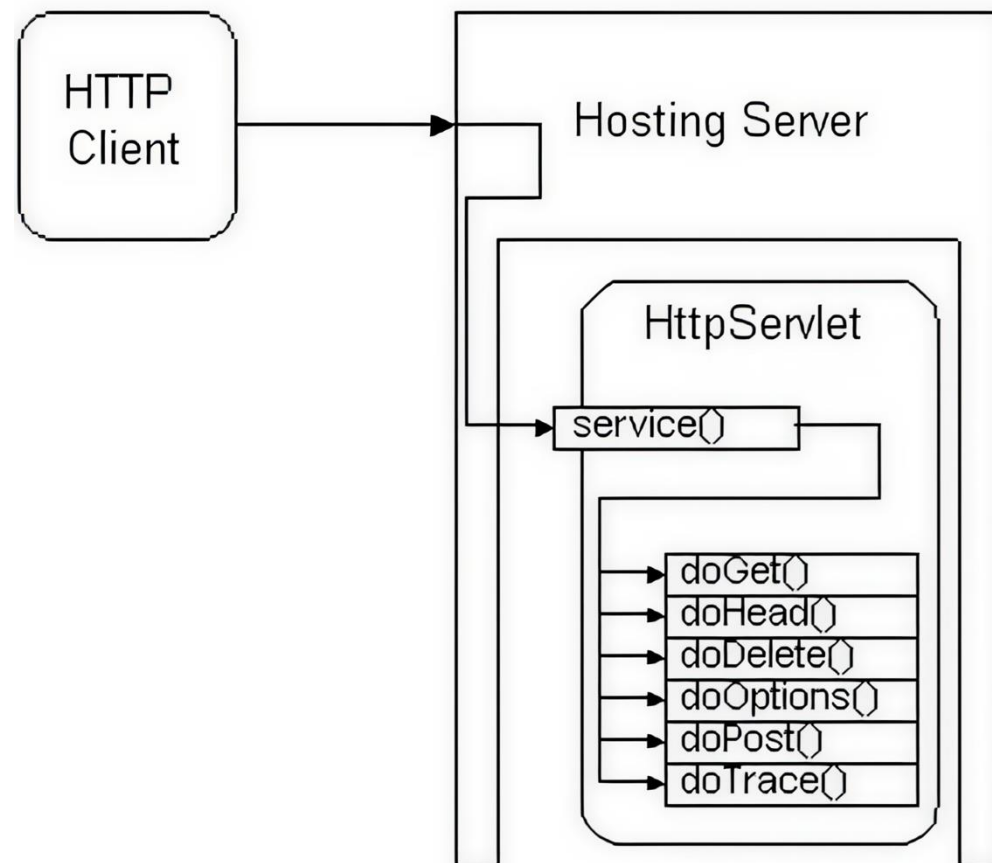
`HttpServletRequest`

`HttpServletResponse`

Eccezioni

`ServletException`

`IOException`



Richieste e risposte

- Le interfacce `HttpServletRequest` e `HttpServletResponse` consentono di **gestire richieste e risposte HTTP** in modo strutturato.
- Permettono di **accedere ai dati** contenuti nell'**header** e nel **body** dei messaggi HTTP, come parametri, metodi di richiesta, codici di stato, contenuti, ecc.
- **HttpServletRequest**
 - Oggetto passato dal metodo `service()`
 - Rappresenta la **richiesta HTTP** inviata dal client
 - Estende `ServletRequest`
 - Consente di leggere **parametri**, **header**, **metodo di richiesta** (GET, POST), e il **corpo** del messaggio
- **HttpServletResponse**
 - Oggetto passato dal (e restituito al) metodo `service()`
 - Rappresenta la risposta HTTP inviata al client
 - Estende `ServletResponse`
 - Permette di impostare **status code**, **header**, e scrivere nel **corpo** della risposta

Richieste e risposte

GET /SlideServlet/GetPostHTTPServlet?animal=none HTTP/1.1

Query parameter

- I metodi principali per manipolare le richieste (`HttpServletRequest`)

`String getParameter(String name)` - Restituisce il valore del *query parameter* dato il nome (valore singolo)

`Enumeration getParameterNames()` - Restituisce l'elenco dei nomi degli parametri

`String[] getParametersValues(String name)` - Restituisce i valori del parametro **name** (valore multiplo)

- I metodi principali per manipolare le risposte (`HTTPServletResponse`)

`void setContentType(String type)` - Specifica il tipo MIME della risposta per dire al browser come visualizzare la risposta Es: "text/html" dice che è html

`ServletOutputStream getOutputStream()` - Restituisce lo stream di byte per scrivere nel corpo della risposta

`PrintWriter getWriter()` - Restituisce lo stream di caratteri per scrivere nel corpo della risposta

Richieste e risposte

- Altri metodi

`Cookie[] getCookies()` - Restituisce i cookies del server sul client

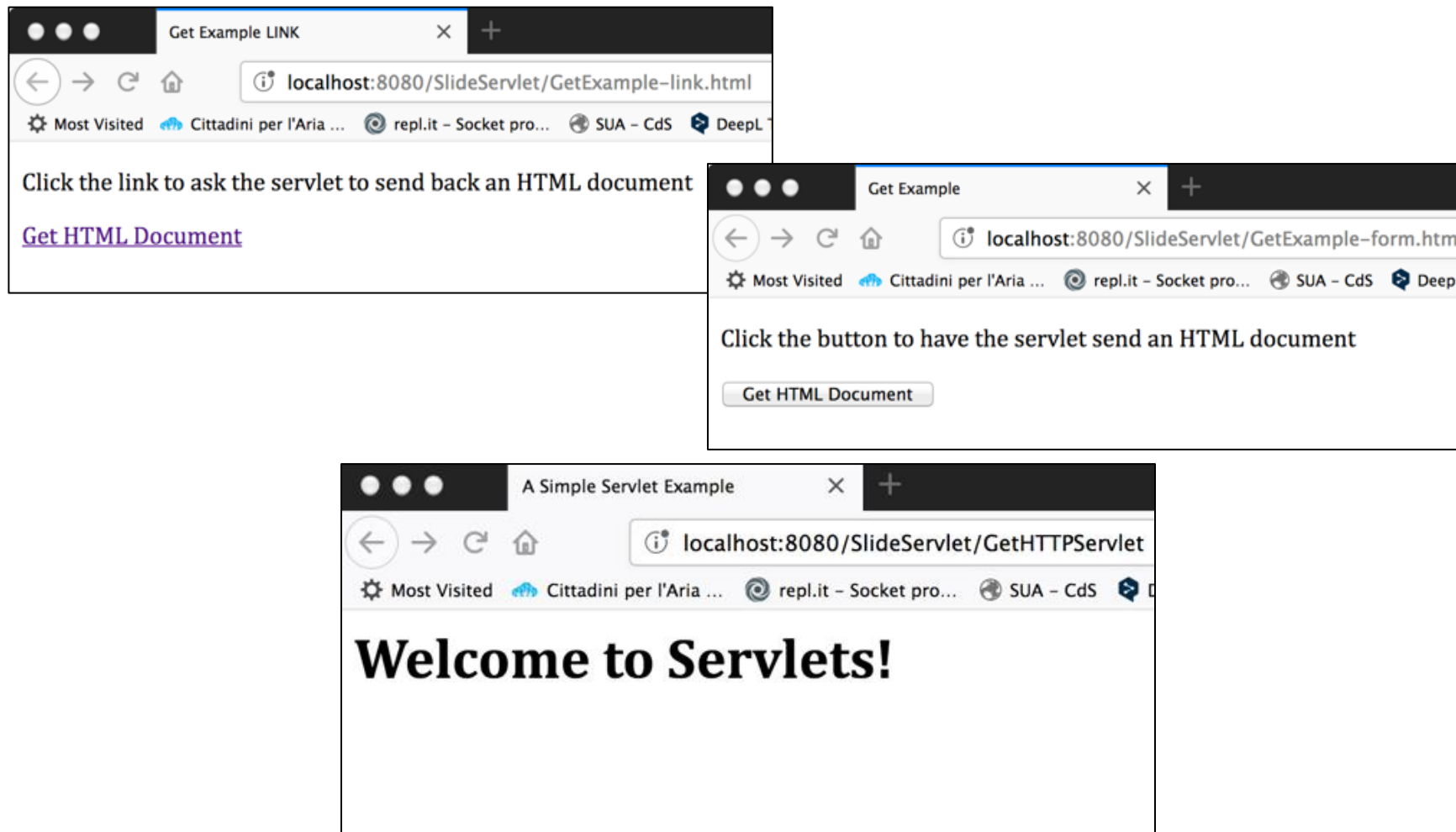
`void addCookie(Cookie cookie)` - Aggiunge un cookie nell'intestazione (header) della risposta

`HTTPSession getSession(boolean create)` - Una `HTTPSession` identifica il client.

Viene creata se `create=true`



Un esempio GET

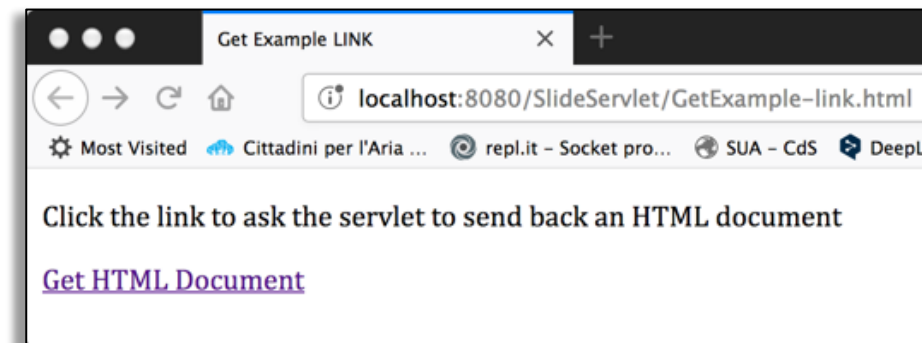


Lato client: pagina HTML con link

```
1. <html>
2. <head>
3. <meta charset="UTF-8">
4. <title>Get Example LINK</title>
5. </head>
6. <body>

7.   <p>Click the link to ask the servlet to send back an HTML document</p>
8.   <a href="http://localhost:8080/SlideServlet/GetHTTPServlet" >
9.       Get HTML document
10.  </a>

11.</body>
12.</html>
```



URL riferita alla *risorsa* servlet

Lato client: pagina HTML con Form

```

1. <html>
2. <head>
3. <meta charset="UTF-8">
4. <title>Get Example</title>
5. </head>
6. <body>

7. <form action="http://localhost:8080/SlideServlet/GetHTTPServlet" method="GET">
8.     <p>Click the link to ask the servlet to send back an HTML document</p>
9.     <input type="submit" value="Get HTML Document">
10.</form>

11.</body>
12.</html>

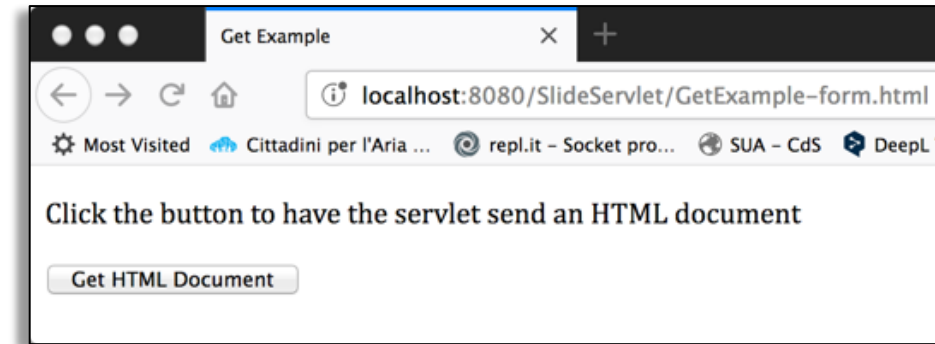
```

L'applicazione che processa il form

Metodo di invio

Crea il bottone di invio

Etichetta esposta



Lato server: servlet con metodo doGet

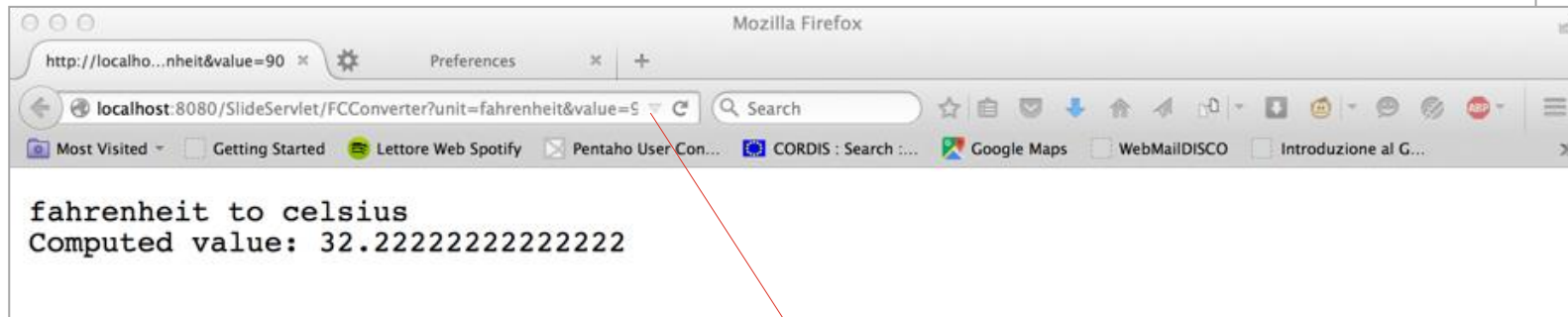
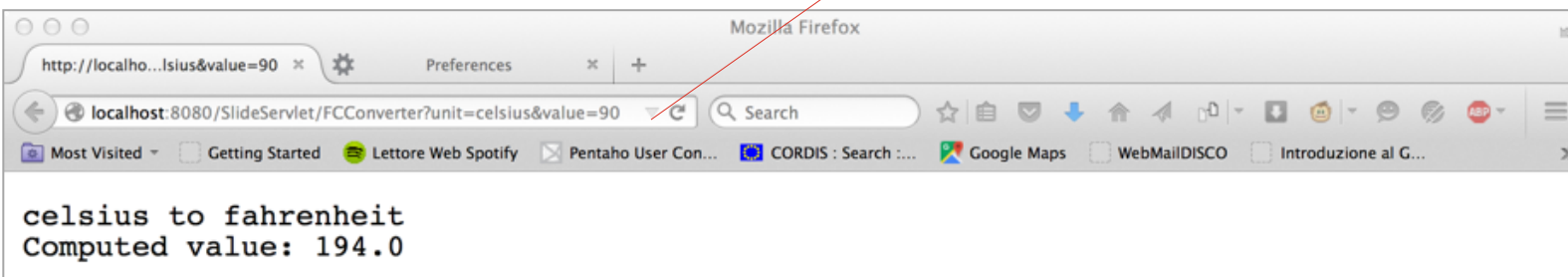
```
1. // da Internet e WWW - How to program, Dietel&Dietel, Prentice Hall
2. // Creating and sending a page to the client
3. public class GetHTTPServlet extends HttpServlet {
4.     void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException,
        IOException {
5.         PrintWriter output;
6.         response.setContentType( "text/html" );    // content type
7.         output = response.getWriter();              // get writer
8.         // create and send HTML page to client
9.         StringBuffer buf = new StringBuffer();
10.        buf.append( "<HTML><HEAD><TITLE>\n" );
11.        buf.append( "A Simple Servlet Example\n" );
12.        buf.append( "</TITLE></HEAD><BODY>\n" );
13.        buf.append( "<H1>Welcome to Servlets!</H1>\n" );
14.        buf.append( "</BODY></HTML>" );
15.        output.println( buf.toString() );
16.        output.close();    // close PrintWriter stream
17.    }
18.}
```



Un esempio - GET con parametri

- Convertitore da gradi Celsius a Fahrenheit e viceversa

<http://localhost:8080/SlideServlet/FCConverter?unit=celsius&value=90>



<http://localhost:8080/SlideServlet/FCConverter?unit=fahrenheit&value=90>

Lato server: il metodo doGet

```
1. public class FCConverter extends HttpServlet {
2.     private String convertCtoF(Double celsius) {
3.         Double fahrenheit; fahrenheit = ((celsius * 9) / 5) + 32; return Double.toString(fahrenheit);
4.     }
5.     private String convertFtoC(Double fahrenheit) {
6.         Double celsius; celsius = (fahrenheit - 32)*5/9; return Double.toString(celsius);
7.     }

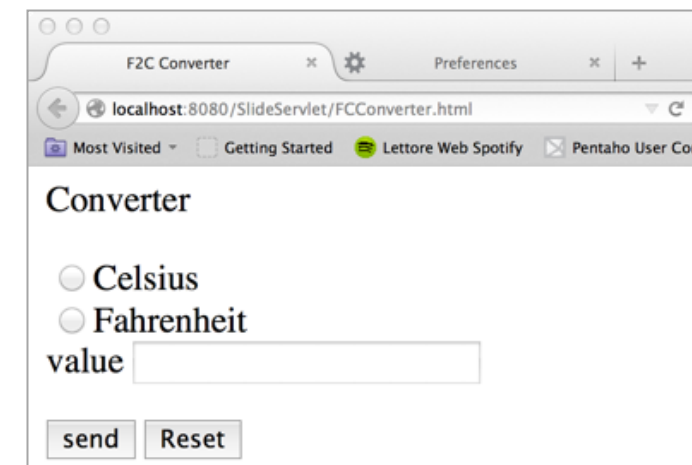
8.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
9.         throws ServletException, IOException {
10.         String unit=request.getParameter("unit");
11.         Double value=Double.parseDouble(request.getParameter("value"));
12.         String result;
13.         if( unit.equals("fahrenheit")) {
14.             response.getWriter().append("fahrenheit to celsius \n");
15.             result=convertFtoC(value);
16.         } else {
17.             response.getWriter().append("celsius to fahrenheit \n");
18.             result=convertCtoF(value);
19.         }
20.         response.getWriter().append("Computed value: ").append(result);
21.     }
22. }
```

FC converter con Form HTML

```
1. <html>
2. <head>
3. <meta charset="UTF-8">
4. <title>F2C Converter</title>
5. </head>
6. <body>

7. <form action="http://localhost:8080/SlideServlet/FCConverter"
8.     method="GET">
9.     Converter <br><br>
10.    <input type="radio" name="unit" value="celsius">Celsius<br>
11.    <input type="radio" name="unit" value="fahrenheit">Fahrenheit<br>
12.    value <input type="text" name="value" ><br><br>
13.    <input type="submit" value="send">
14.    <input type="reset">
15.</form>

16.</body>
17.</html>
```



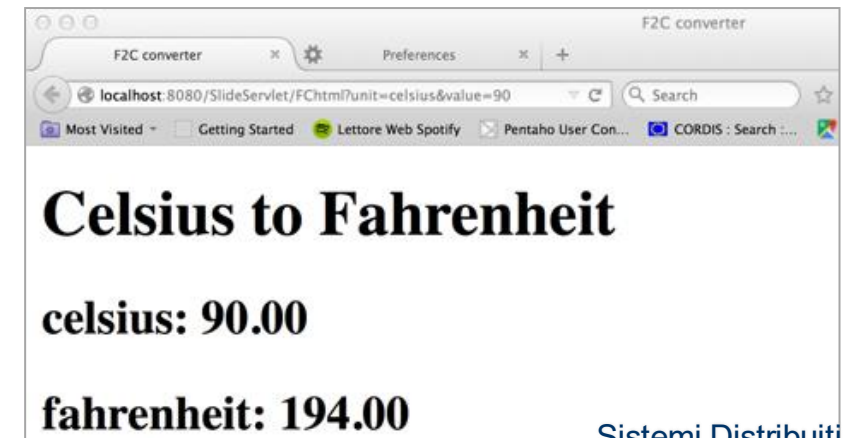
The screenshot shows a web browser window with the title "F2C Converter". The address bar displays "localhost:8080/SlideServlet/FCConverter.html". The browser's bookmark bar includes "Most Visited", "Getting Started", "Lettore Web Spotify", and "Pentaho User Con.". The main content area is titled "Converter" and contains two radio buttons: "Celsius" (selected) and "Fahrenheit". Below the radio buttons is a text input field labeled "value". At the bottom of the form are two buttons: "send" and "Reset".

Servlet che restituisce HTML

```
1. public class Fhtml extends HttpServlet {
2.     private Double convertCtoF(Double celsius) { // restituisce un Double
3.         Double fahrenheit;
4.         fahrenheit = ((celsius * 9) / 5) + 32;
5.         return fahrenheit;
6.     }
7.     private Double convertFtoC(Double fahrenheit) { // restituisce un Double
8.         Double celsius;
9.         celsius = (fahrenheit - 32) * 5 / 9;
10.        return celsius;
11.    }
12.    protected void doGet(HttpServletRequest request, HttpServletResponse response)
13.        throws ServletException, IOException {
14.        ...
15.    }
16. }
```

Servlet che restituisce HTML

```
1. protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
2.     String unit=request.getParameter("unit");
3.     Double value=Double.parseDouble(request.getParameter("value"));
4.     Double result;
5.     StringBuffer buf = new StringBuffer();
6.     buf.append( "<!DOCTYPE html>\n" );
7.     buf.append( "<html>\n<head>\n<title>F2C converter </title>\n</head>\n<body>\n" );
8.     if( unit.equals("fahrenheit")) {
9.         buf.append("<h1> Fahrenheit to Celsius </h1>\n"); result=convertFtoC(value);
10.    } else {
11.        buf.append("<h1> Celsius to Fahrenheit </h1>\n"); result=convertCtoF(value);
12.    }
13.    DecimalFormat twoDigits = new DecimalFormat( "#0.00" );
14.    buf.append( "<h2>" + unit + ": " + twoDigits.format( value ) + "</h2>\n" ); buf.append( "<h2>" );
15.    if( unit.equals("fahrenheit")) {
16.        buf.append("celsius: ");
17.    } else {
18.        buf.append("fahrenheit: ");
19.    }
20.    buf.append( twoDigits.format( result ) );
21.    buf.append( "</h2>\n" ); buf.append( "</body>\n</html>" );
22.    response.setContentType( " text/html" );
23.    response.getWriter().append(buf.toString());
24.}
```



Converter con Form e risposta HTML

The image displays three sequential browser window screenshots illustrating the development of a web-based temperature converter.

First Screenshot: The browser window shows the initial form at `localhost:8080/SlideServlet/FChtml.html`. The form is titled "Converter" and contains two radio buttons: "Celsius" (selected) and "Fahrenheit". Below the radio buttons is a text input field labeled "value" and two buttons: "send" and "Reset".

Second Screenshot: The browser window shows the form after the "send" button has been clicked. The "Celsius" radio button remains selected, and the "value" field now contains the number "90".

Third Screenshot: The browser window shows the result page at `localhost:8080/SlideServlet/FChtml?unit=celsius&value=90`. The page displays the title "Celsius to Fahrenheit" and the converted values: "celsius: 90.00" and "fahrenheit: 194.00".

Configurazione di una servlet

Le servlet richiedono una configurazione che definisca:

- Nome della servlet
- Classe di implementazione
- Mapping URL
- Parametri di inizializzazione

Due approcci di configurazione:

1. tramite *web.xml* (Deployment Descriptor)
2. Tramite annotazioni (Servlet 3.0+)

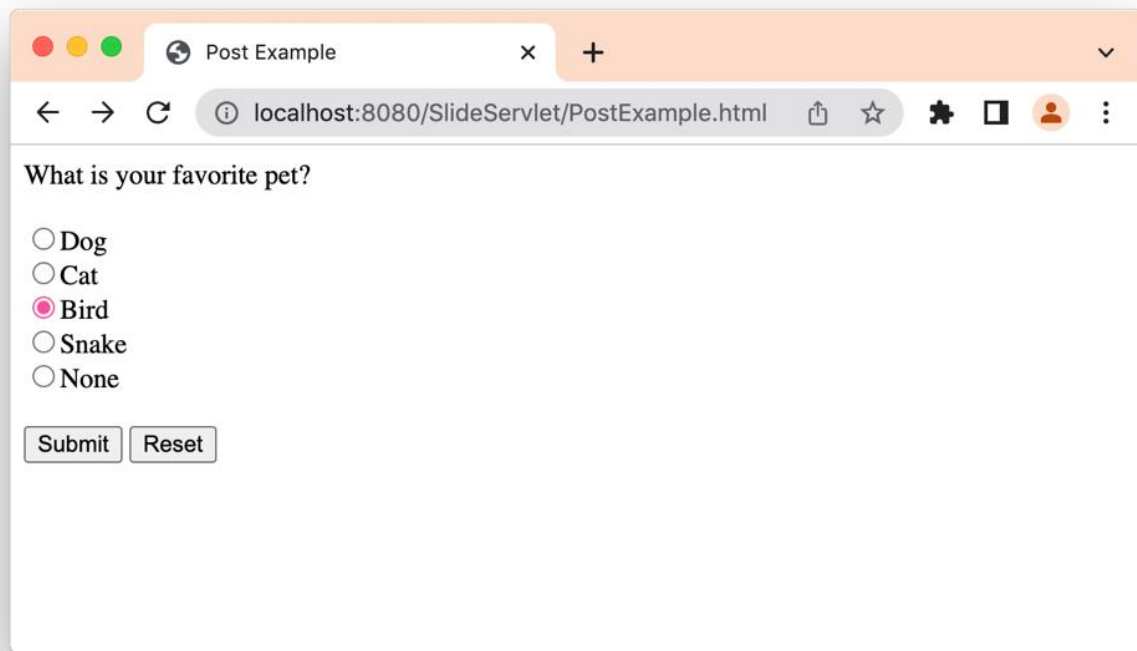
```
<servlet>
  <servlet-name>Fhtml</servlet-name>
  <servlet-class>com.example.Fhtml</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>Fhtml</servlet-name>
  <url-pattern>/SlideServlet/FCConverter</url-pattern>
</servlet-mapping>
```

```
@WebServlet("/SlideServlet/FCConverter")
public class Fhtml extends HttpServlet {
    // implementazione
}
```

In Java, le **annotazioni** sono elementi sintattici preceduti da **@** che aggiungono **metadati** a classi, metodi, variabili, parametri o altri elementi del codice. Queste informazioni possono essere usate dal compilatore, da strumenti di sviluppo o da framework attivare comportamenti specifici **a tempo di esecuzione**.

Un esempio con POST

- Sondaggio: qual è l'animale da compagnia preferito?



Post Example

localhost:8080/SlideServlet/PostExample.html

What is your favorite pet?

☐ Dog

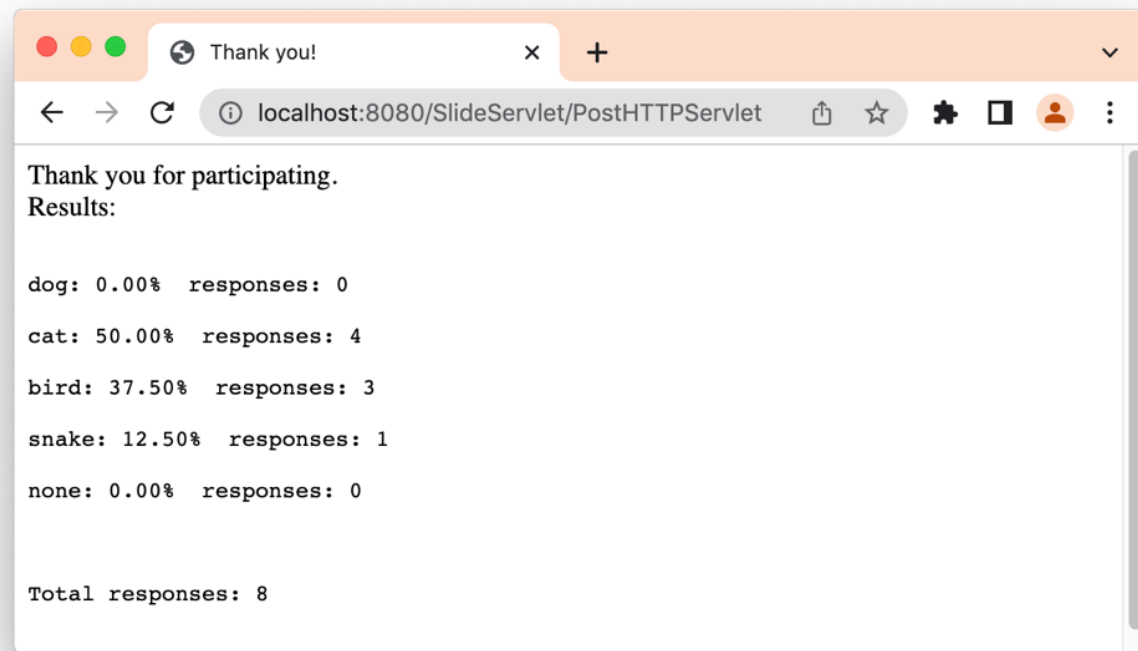
☐ Cat

☒ Bird

☐ Snake

☐ None

Submit Reset



Thank you!

localhost:8080/SlideServlet/PostHTTPServlet

Thank you for participating.
Results:

```
dog: 0.00%  responses: 0
cat: 50.00%  responses: 4
bird: 37.50%  responses: 3
snake: 12.50%  responses: 1
none: 0.00%  responses: 0
```

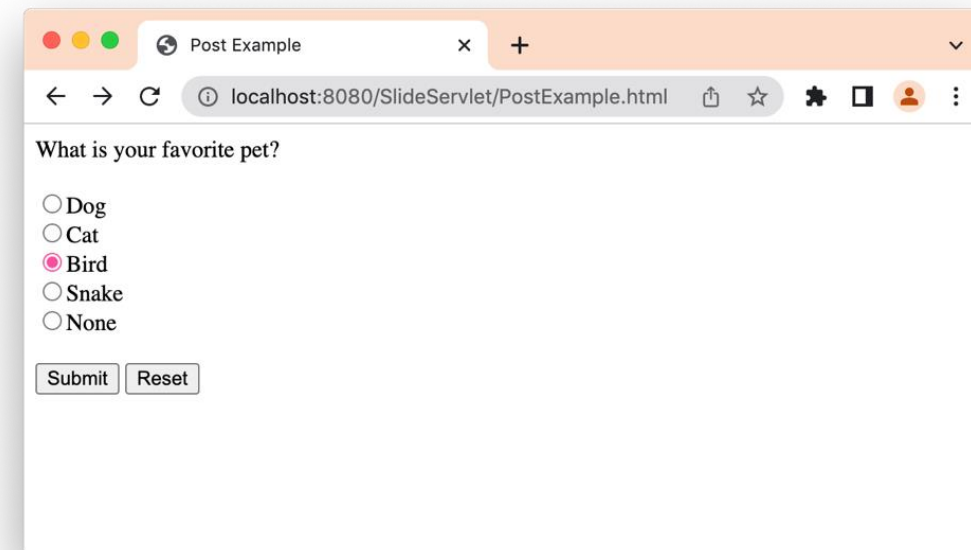
Total responses: 8

Lato client: la pagina HTML

```
1. <html>
2. <head>
3. <meta charset="UTF-8">
4. <title>Post Example</title>
5. </head>
6. <body>

7.   <form action="http://localhost:8080/SlideServlet/PostHTTPServlet" method="POST">
8.     What is your favorite pet?<br>
9.     <br> <input type="radio" name="animal" value="dog">Dog<br>
10.    <input type="radio" name="animal" value="cat">Cat<br> <input
11.      type="radio" name="animal" value="bird">Bird<br> <input
12.      type="radio" name="animal" value="snake">Snake<br> <input
13.      type="radio" name="animal" value="none" checked>None <br><br>
14.    <input type="submit" value="Submit"> <input type="reset">
15.  </form>

16.</body>
17.</html>
```



Lato server: il metodo doPost

```
1.  public class PostHTTPServlet extends HttpServlet {
2.      // definisco l'elenco degli animali
3.      private String animalNames[] = { "dog", "cat", "bird", "snake", "none" };
4.      private int[5] animalCounts = initializeToZero(); // contatori di preferenze
5.      protected void doPost(HttpServletRequest request, HttpServletResponse response) {
6.          int total = 0; // totale delle preferenze espresse
7.          // i dati sono memorizzati nel file "survey.dat"
8.          File file = new File("survey.dat"); // apro o creo il file
9.          if (file.exists()) {
10.             ObjectInputStream input = new ObjectInputStream(new FileInputStream(file));
11.             try {
12.                 animalCounts = (int[]) input.readObject(); // leggo il file e lo assegno a animals
13.             } catch (ClassNotFoundException e) {
14.                 e.printStackTrace();
15.             }
16.             // conto quante sono le risposte date in precedenza (salvate sul file)
17.             for (int i = 0; i < animalCounts.length; ++i)
18.                 total += animalCounts[i];
19.         }
```

doPost (continua)

```
22. // leggo il messaggio con la nuova preferenza
23. String value = request.getParameter("animal");
24. ++total; // aggiorno il totale delle risposte

25. // determino quello votato e aggiorno il suo contatore
26. for (int i = 0; i < animalNames.length; ++i)
27.     if (value.equals(animalNames[i]))
28.         ++animalCounts[i];

29. // scrivo i nuovi contatori sul file e lo chiudo
30. try (ObjectOutputStream output = new ObjectOutputStream(new FileOutputStream(file))) {
31.     output.writeObject(animalsCounts);
32. }

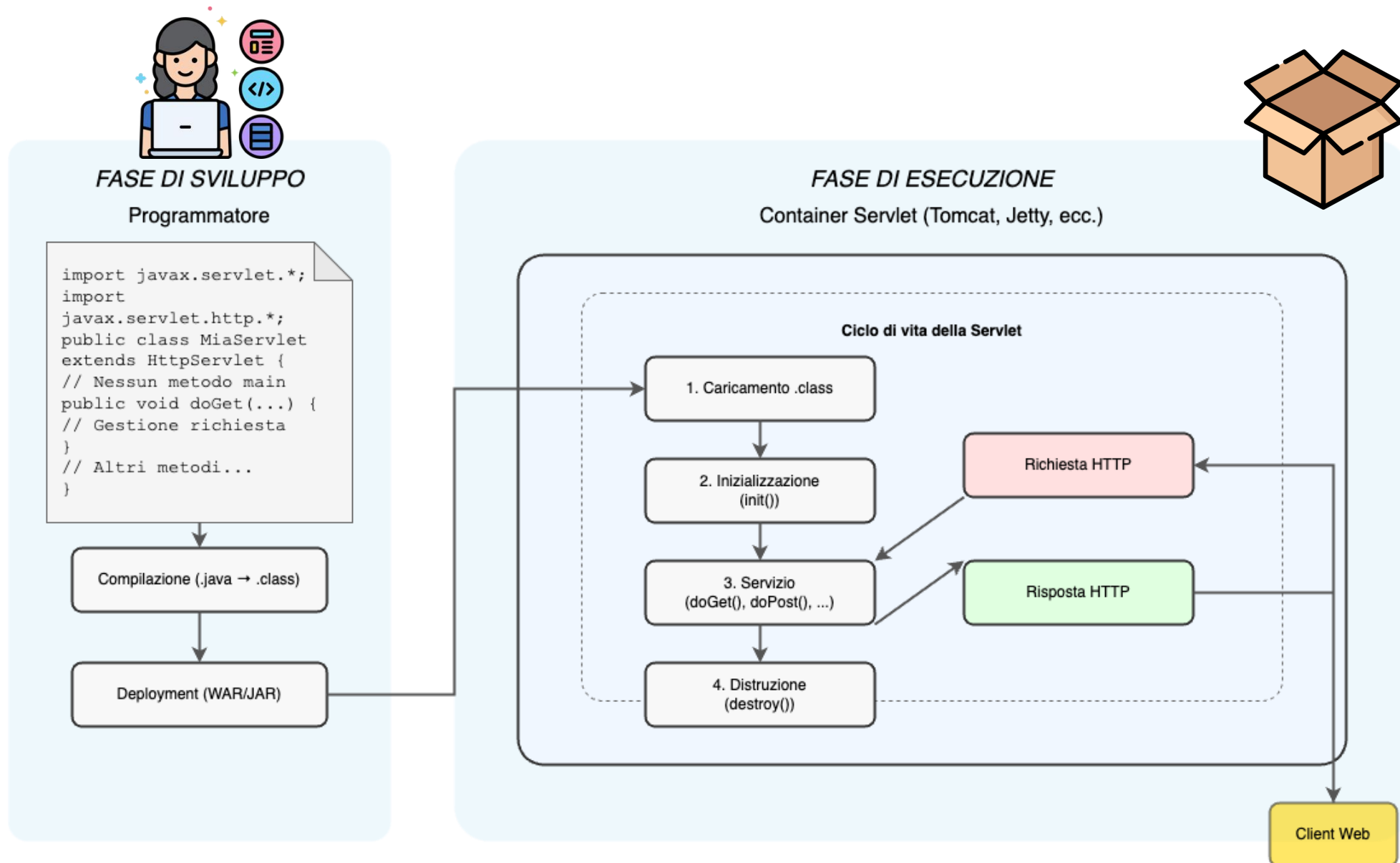
33. // calcolo le percentuali
34. double percentages[] = new double[animalCounts.length];
35. for (int i = 0; i < percentages.length; ++i)
36.     percentages[i] = 100.0 * animalCounts[i] / total;
```

doPost (continua)

```
38. // uso un Buffer di servizio per costruire la pagina html in risposta
39. StringBuffer buf = new StringBuffer();
40. buf.append("<html>\n<title>Thank you!</title>\n");
41. buf.append("Thank you for participating.\n");
42.   buf.append("<br>Results:\n<pre>");
43. DecimalFormat twoDigits = new DecimalFormat("#0.00");
44. for (int i = 0; i < percentages.length; ++i) {
45.     buf.append("<br>" + animalNames[i] + ": " + twoDigits.format(percentages[i]));
46.     buf.append("%   responses: " + animalCounts[i] + "\n");
47. }
48. buf.append("\n<br><br>Total responses: " + total);
49. buf.append("</pre>\n</html>");

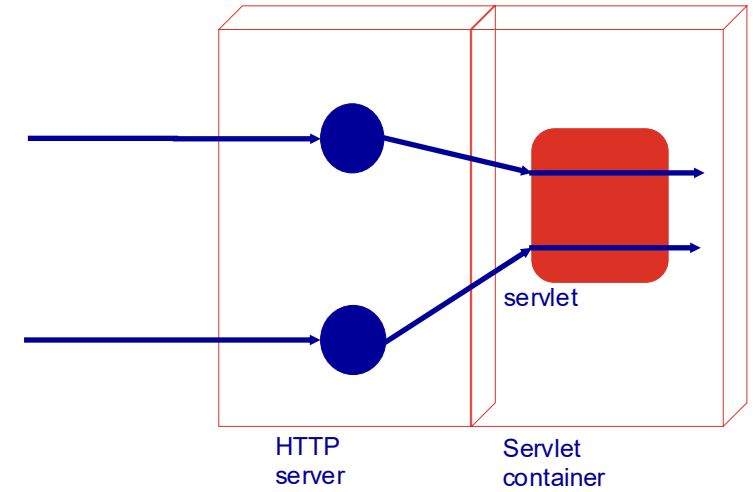
50. // costruisco l'head del messaggio di risposta
51. response.setContentType("text/html"); // dichiaro il formato del body
52. // predispongo il canale stream per la scrittura del body del messaggio
53. PrintWriter responseOutput = response.getWriter();
54. // scrivo la pagina nella risposta
55. responseOutput.println(buf.toString());
56. }
57. }
```

Ciclo di vita di una Servlet

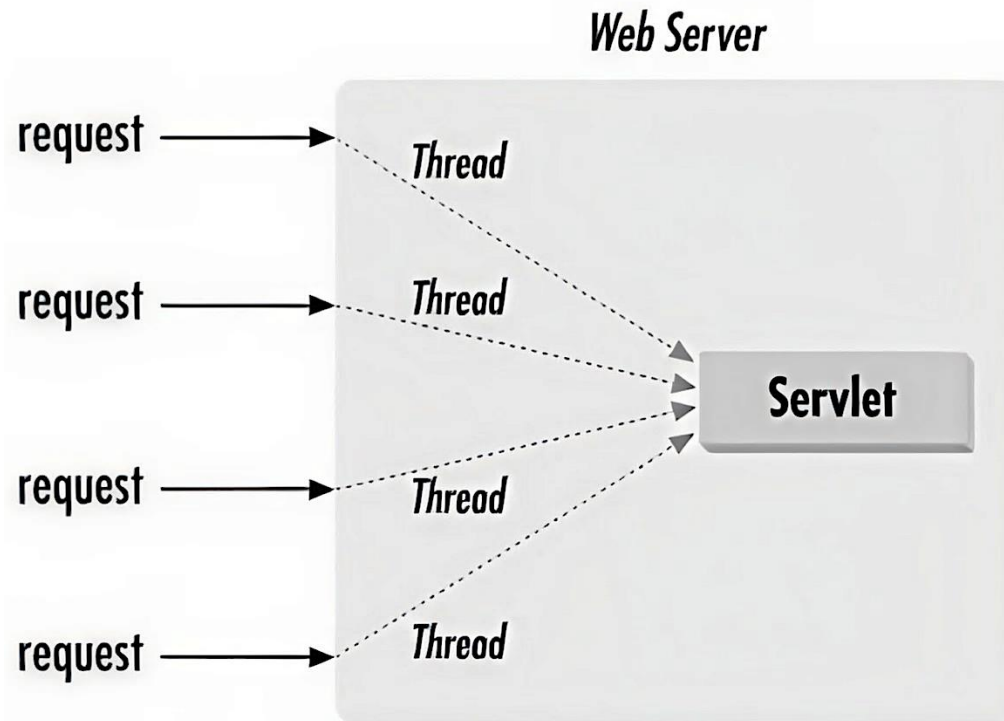


Ciclo di vita di una servlet

- Una servlet viene creata dal container/engine
 - Quando viene effettuata la prima chiamata da parte del client
 - La servlet viene condivisa da tutti client. Più thread 1 servlet.
 - Ogni richiesta genera un **Thread** che esegue la **doXXX** appropriata
- Il container/engine invoca il metodo **init()** per inizializzazioni specifiche
- Una servlet viene distrutta dall'engine all'occorrenza di uno dei due eventi:
 - Quando non ci sono thread in esecuzione su quella servlet
 - Quando è scaduto un timeout predefinito
- Viene invocato il metodo **destroy()** per terminare correttamente la servlet



Servlet e Thread



https://docstore.mik.ua/oreilly/java-ent/servlet/ch03_01.htm

- Molti thread -> una singola istanza della servlet
- Più richieste vengono servite dalla stessa servlet, questo vuol dire che bisogna far attenzione a come la servlet implementa l'accesso alle sue risorse
- In questo modello spesso le risorse sono condivise a livello di database

Questa scelta ha diverse motivazioni:

- Utilizzare meno memoria (un solo oggetto)
- Ridurre il costo di gestione (per esempio tempi di inizializzazione) di molti oggetti che sarebbero spesso identici
- Abilita la persistenza (in memoria) di risorse condivisibili tra diverse richieste (e.g. una connessione a DB, oggetto "carrello")

Terminazione

- Container e richieste dei client devono sincronizzarsi sulla terminazione
 - Alla scadenza del *timeout* potrebbe essere ancora dei thread in esecuzione in **service ()**
- Bisogna
 - Tener traccia dei *thread* in esecuzione
 - Progettare il metodo **destroy ()** in modo da notificare lo shutdown e attendere il completamento del metodo **service ()**
 - Progettare i metodi lunghi in modo che verifichino periodicamente se è in corso uno shutdown e comportarsi di conseguenza

Realizzazione di applicazioni

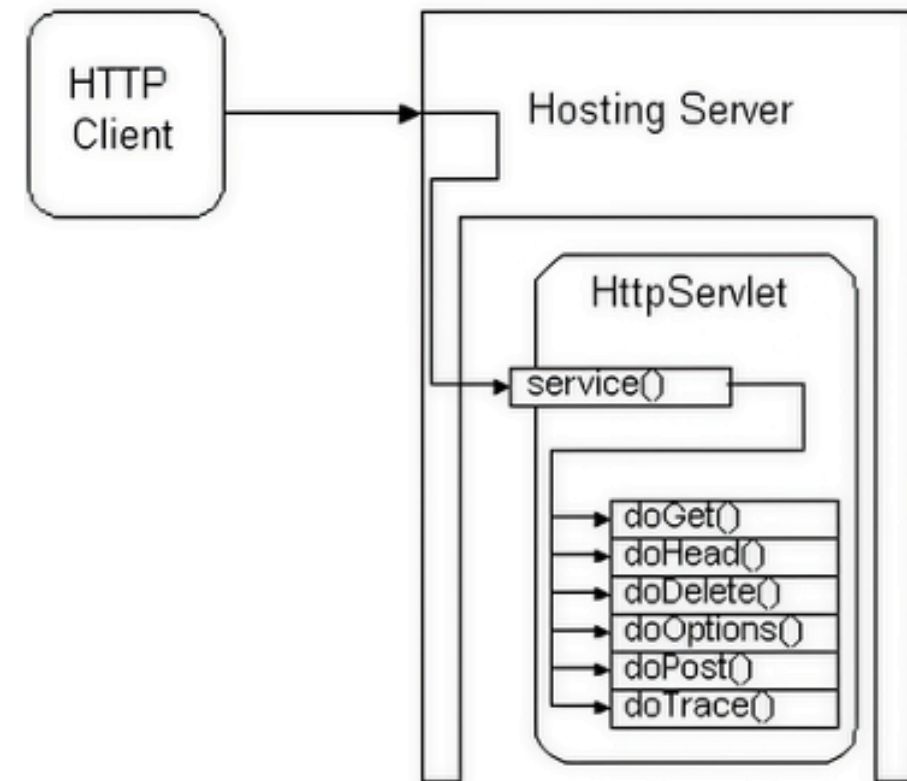
- Servlets «classiche», uso dell'ereditarietà
 - Supporta tutti i metodi
- Spring MVC
 - Spring@MVC, dalla versione 2.5 gestisce tutti i metodi HTTP
 - Usa delle annotazioni per assegnare i metodi

In Java, le **annotazioni** sono elementi sintattici preceduti da @ che aggiungono **metadati** a classi, metodi, variabili, parametri o altri elementi del codice. Queste informazioni possono essere usate dal compilatore, da strumenti di sviluppo o da framework come **Spring** per attivare comportamenti specifici **a tempo di esecuzione**.

```
@Controller
public class BookingsController {

    @RequestMapping(value="/bookings", method=RequestMethod.GET)
    public String getBookings() {
        return "someView";
    }

    @RequestMapping(value="/bookings", method=RequestMethod.POST)
    public void addBooking(HttpServletRequest request) {
        // read request
    }
}
```



Realizzazione di applicazioni

- JAX-RS (e.g., jersey framework)
 - JSR 311
 - Framework REST basato su annotazioni

```
@Path("/hotels")
public class HotelsResource {

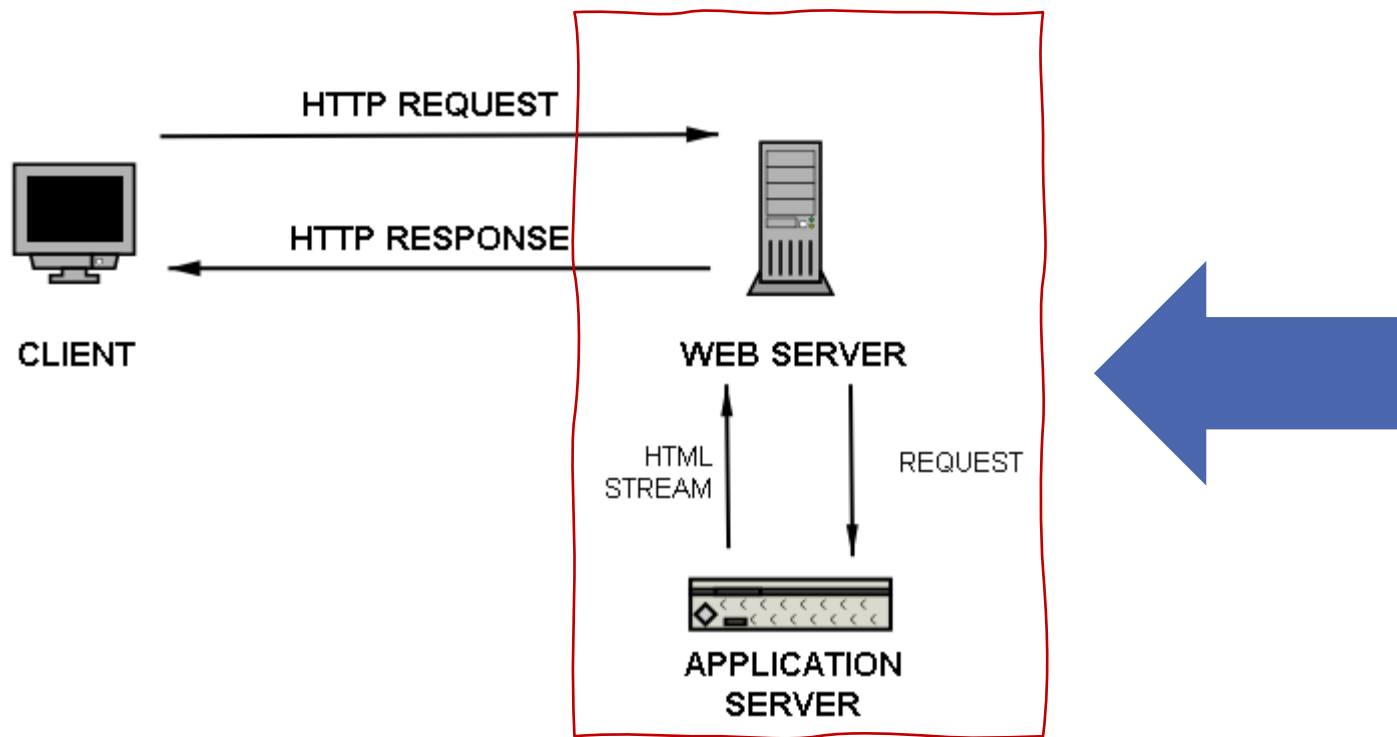
    @GET
    @Produces("application/xml")
    public HotelList getHotels(
        @QueryParam("searchString") String searchString) {
        // do something with query parameter
    }

    @Path("{hotelId}")
    public HotelResource getHotel() {
        return new HotelResource();
    }

}
```

<https://jersey.java.net/documentation/latest/getting-started.html>

Annotation	Description
@PATH(your_path)	Sets the path to base URL + /your_path. The base URL is based on your application name, the servlet and the URL pattern from the web.xml" configuration file.
@POST	Indicates that the following method will answer to a HTTP POST request
@GET	Indicates that the following method will answer to a HTTP GET request
@PUT	Indicates that the following method will answer to a HTTP PUT request
@DELETE	Indicates that the following method will answer to a HTTP DELETE request
@Produces(MediaType.TEXT_PLAIN [, more-types])	@Produces defines which MIME type is delivered by a method annotated with @GET. In the example text ("text/plain") is produced. Other examples would be "application/xml" or "application/json".
@Consumes(type [, more-types])	@Consumes defines which MIME type is consumed by this method.
@PathParam	Used to inject values from the URL into a method parameter. This way you inject for example the ID of a resource into the method to get the correct object.



SERVER SIDE - JSP



Java Server Pages

- È una tecnologia per la creazione di applicazioni web.
- Specifica l'interazione tra un contenitore/server ed un insieme di documenti (**pagine**) che presentano informazioni all'utente (**viste**)
- Le pagine sono costituite da tag tradizionali (HTML, XML, WML, ...) e da tag applicativi che controllano la generazione del contenuto (generazione server-side)
- JSP, rispetto alle servlet, sono pensate per facilitare la separazione tra logica applicativa e presentazione
- Analogo alla tecnologia Microsoft Active Server Page (ASP)
- Differenze
 - una Java Server Page fa riferimento ad un programma Java eseguito sul Web server
 - una Active Server Page contiene uno script VBScript o JScript

Java Server Pages

- Java Server Pages (JSP) separano la parte dinamica delle pagine dal template HTML statico
 - Il codice JSP va incluso in tag delimitati da "<%>" e "%>".

- Esempio

- una pagina che visualizza

Grazie per la scelta di Internet Guida Pratica

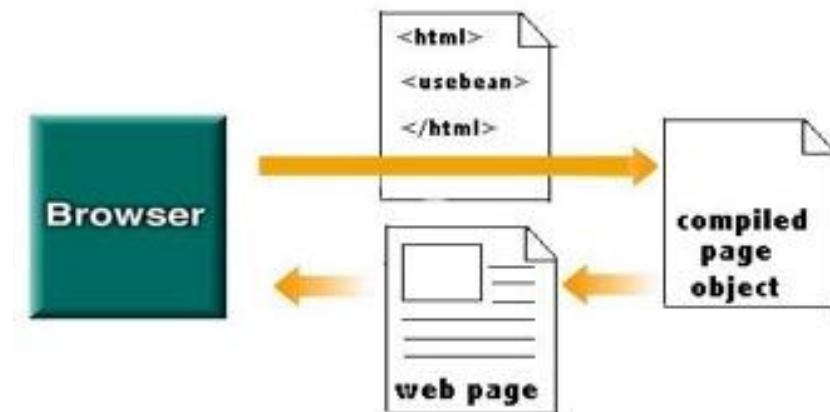
- quando l'utente si connette all'URL

`http://host/OrderConfirmation.jsp?title=Internet+Guida+Pratica`

- la JSP contiene

Grazie per la scelta di `<i> <%= request.getParameter("title") %> </i>`

- La pagina viene convertita automaticamente in una servlet java la prima volta che viene richiesta



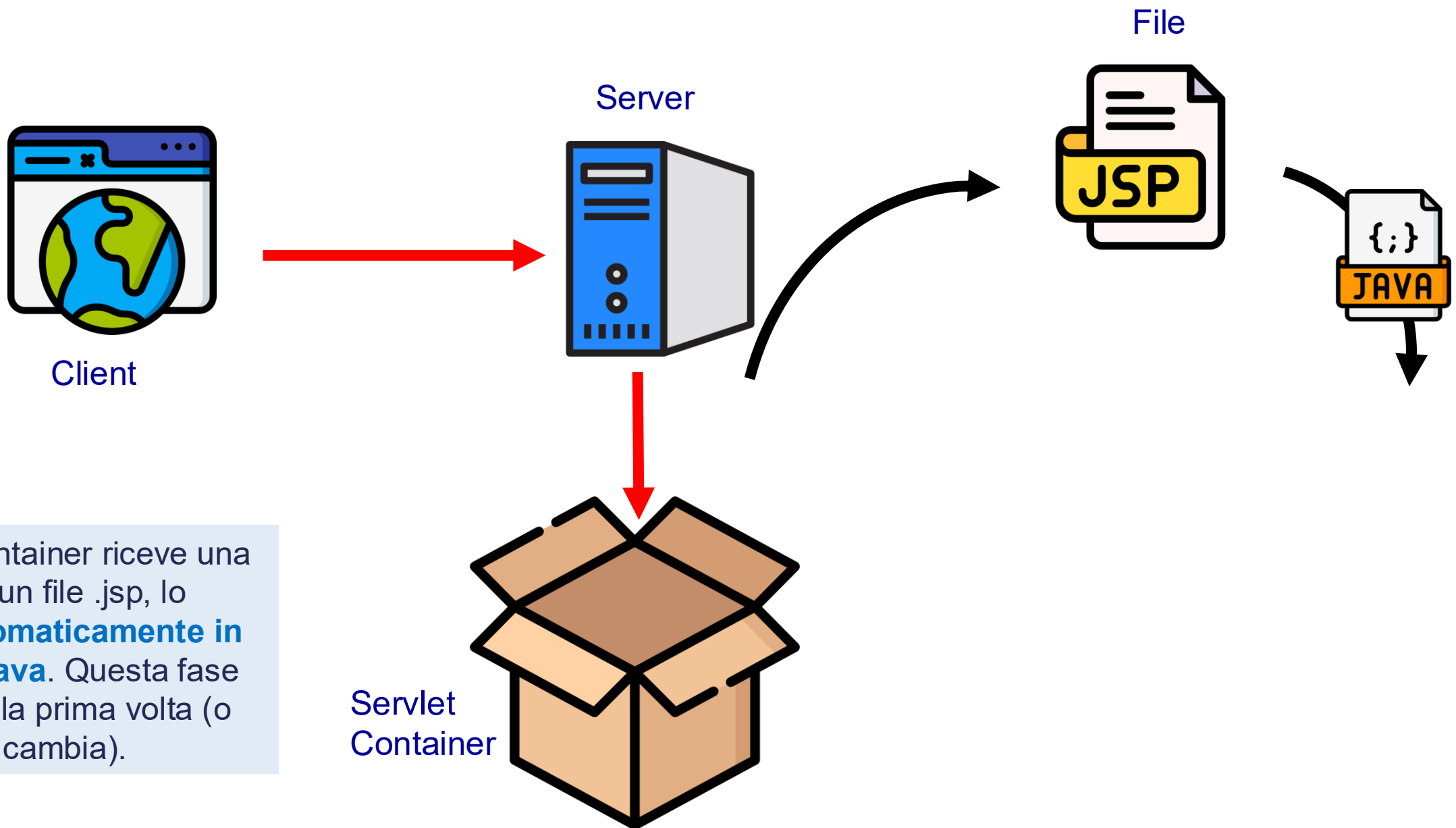
Attenzione: in questo scenario dinamico significa che il contenuto informativo viene inserito dalla JSP a runtime e lato server, quando riceve la richiesta.

Esempio Minimale

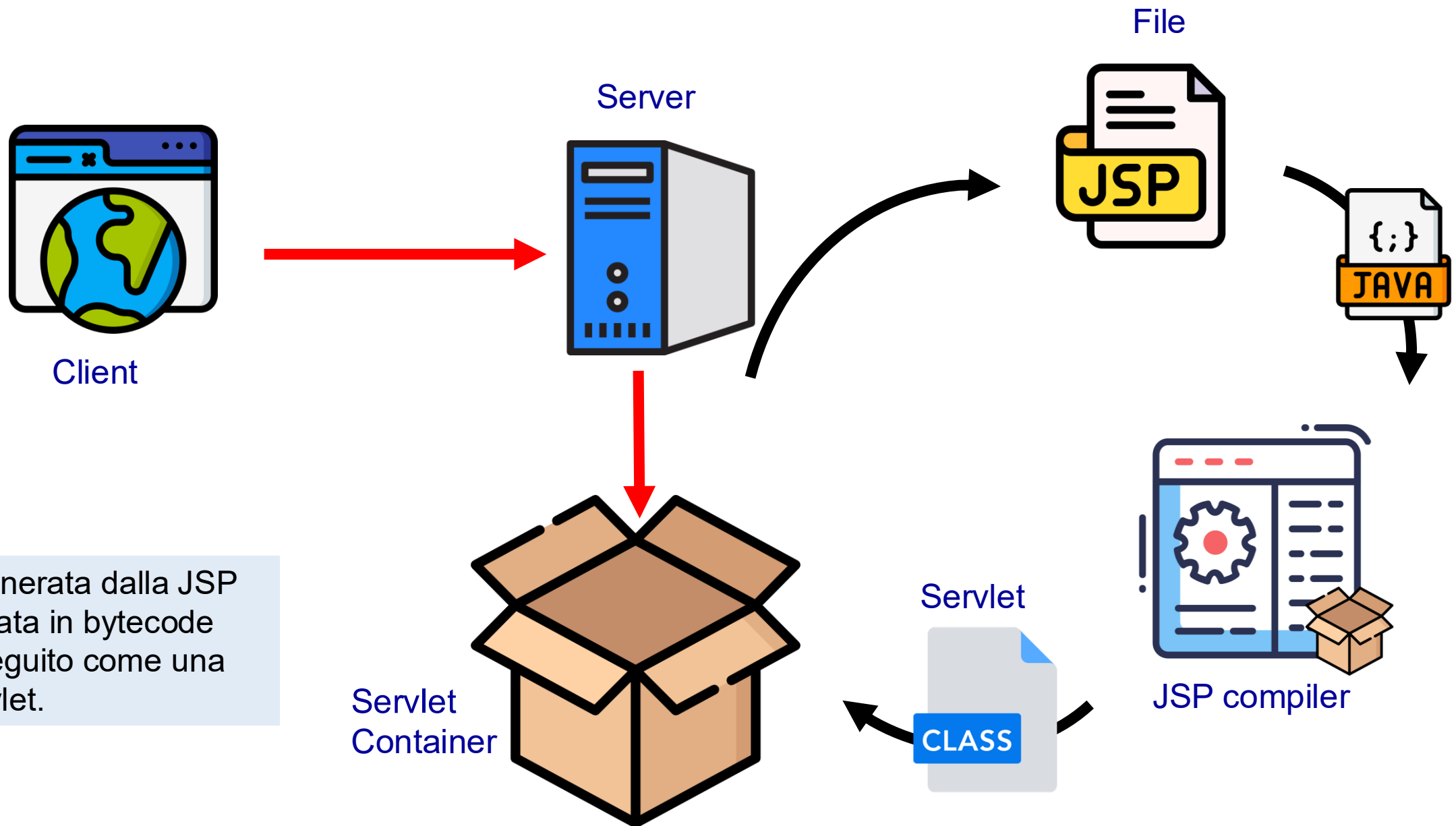
```
1  <%@ page language="java" contentType="text/html;  
2  charset=UTF-8" pageEncoding="UTF-8"%>  
3  <!DOCTYPE html>  
4  <html>  
5  <head>  
6  |   <title>JSP Esempio</title>  
7  </head>  
8  <body>  
9  |   <h1>Esempio JSP Compatto</h1>  
10 |   <%  
11 |       String nome = "Utente";  
12 |       int numero = 5;  
13 |   %>  
14 |   <p>Ciao, <%= nome %>!</p>  
15 |   <ul>  
16 |       <% for(int i = 1; i <= numero; i++) { %>  
17 |       |   <li>Elemento <%= i %></li>  
18 |       <% } %>  
19 |   </ul>  
20 |  
21 |   <p>Data: <%= new java.util.Date() %></p>  
22 </body>  
23 </html>
```



Da JSP a Servlet

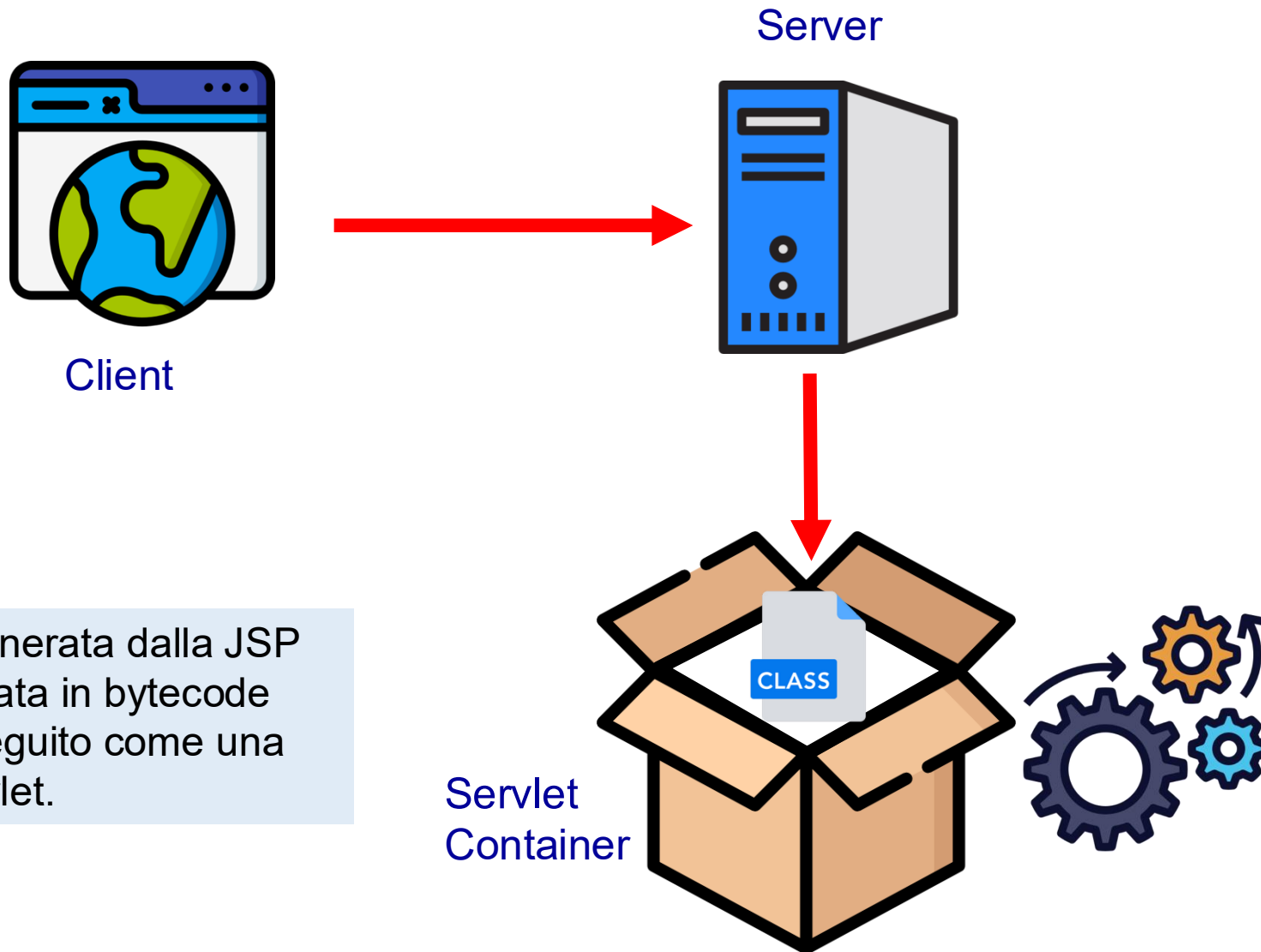


Da JSP a Servlet



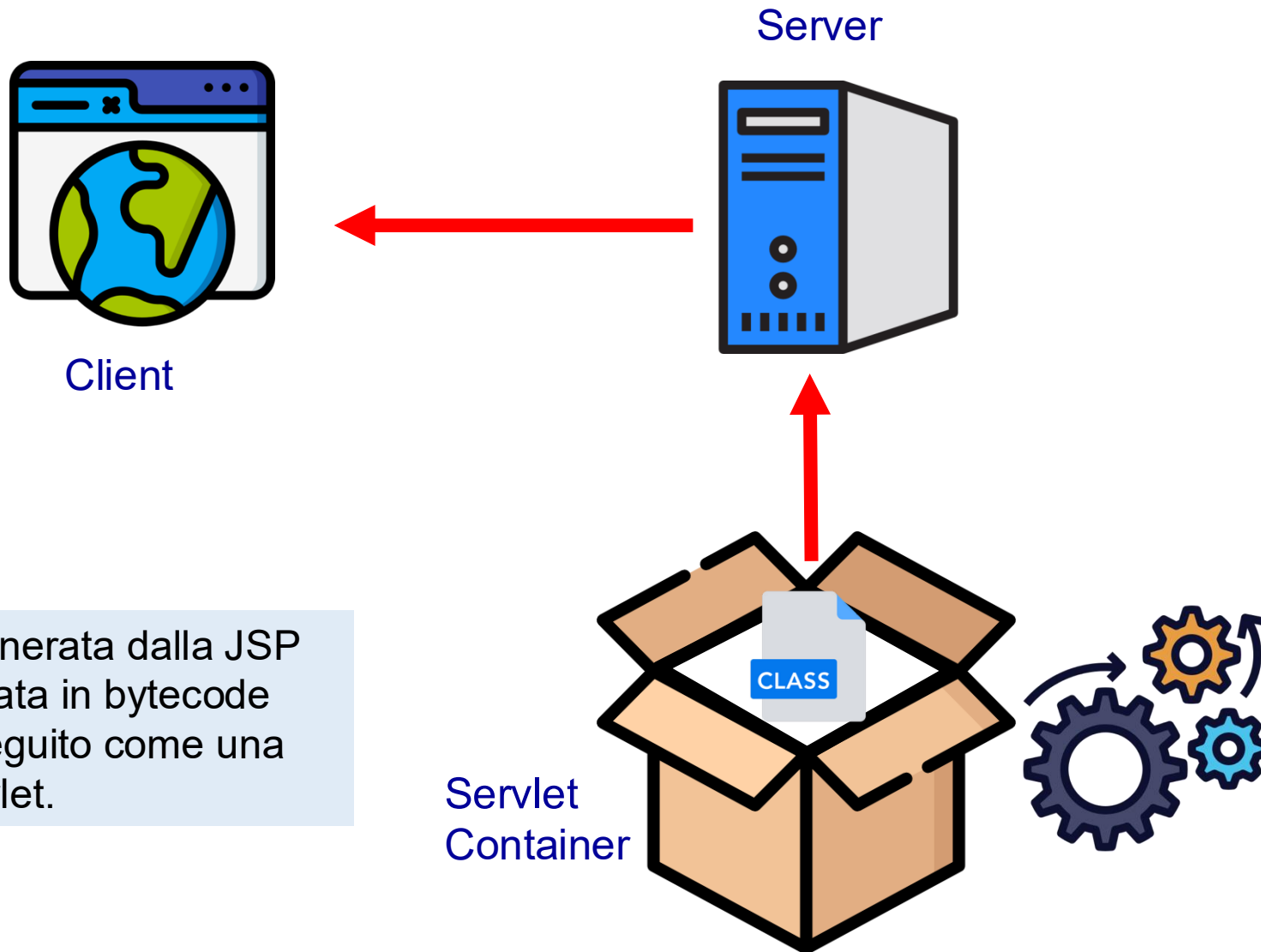
La servlet generata dalla JSP viene compilata in bytecode .class ed eseguito come una normale servlet.

Da JSP a Servlet



La servlet generata dalla JSP viene compilata in bytecode .class ed eseguito come una normale servlet.

Da JSP a Servlet



La servlet generata dalla JSP viene compilata in bytecode .class ed eseguito come una normale servlet.

Gli elementi di una JSP

- **Template text**

- Le parti statiche della pagina HTML

- **Commenti**

`<!-- questo è un commento -->`

- **Direttive**

`<%@ direttiva ... di compilazione %>`

- **Azioni**

`<jsp:XXX attributes> body </jsp:XXX>`

- **Elementi di scripting**

- Istruzioni nel linguaggio specificato nelle direttive
- Sono di tre tipi: *scriptlet*, *declaration*, *expression*

Le **direttive** non influenzano la gestione di una singola richiesta HTTP ma **influenzano le proprietà generali della JSP e come questa deve essere tradotta in una servlet**.

Le **azioni** permettono di supportare diversi comportamenti della pagina JSP. **Vengono processati ad ogni invocazione della pagina JSP**. Permettono di trasferire il controllo da una JSP all'altra, di interagire con i Java Beans, ecc.



Direttive

- **page**

- Liste di attributi/valore
- Valgono per la pagina in cui sono inseriti

```
<%@ page import="java.math.*, java.util.*" %>
```

```
<%@ page session="false" %>
```

- **include**

- Include in compilazione pagine HTML o JSP

```
<%@ include file="copyright.html" %>
```

- **taglib**

- Dichiarare tag definiti dall'utente implementando opportune classi

```
<%@ taglib uri="TableTagLibrary" prefix="table" %>
```

```
<table:loop> ... </table:loop>
```

Le **direttive** non influenzano la gestione di una singola richiesta HTTP ma **influenzano le proprietà generali della JSP e come questa deve essere tradotta in una servlet.**

Tag: il tag nasconde il codice Java dietro un'interfaccia XML. La JSP resta leggibile anche a chi conosce solo HTML (designer, grafici), la logica è scritta una volta nel Tag Handler e riusata in tutte le pagine senza copia-incolla.

Azioni

Le **azioni** permettono di supportare diversi comportamenti della pagina JSP. **Vengono processati ad ogni invocazione della pagina JSP**. Permettono di trasferire il controllo da una JSP all'altra, di interagire con i Java Beans, ecc

- **forward**

- determina l'invio della richiesta corrente, eventualmente aggiornata con ulteriori parametri, alla JSP indicata

```
<jsp:forward page="login.jsp" >
  <jsp:param name="username" value="user" />
  <jsp:param name="password" value="pass" />
</jsp:forward>
```

- **include**

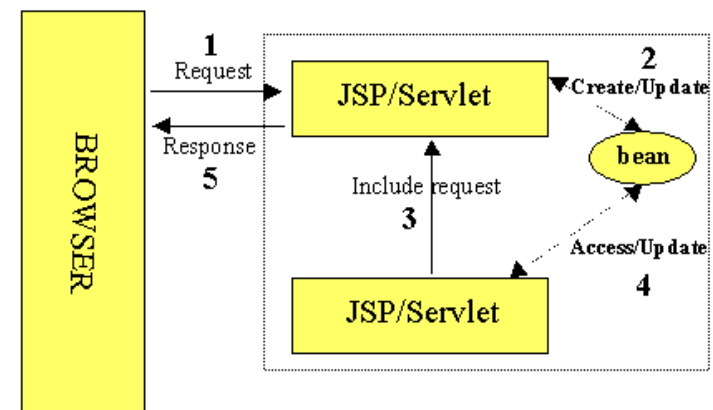
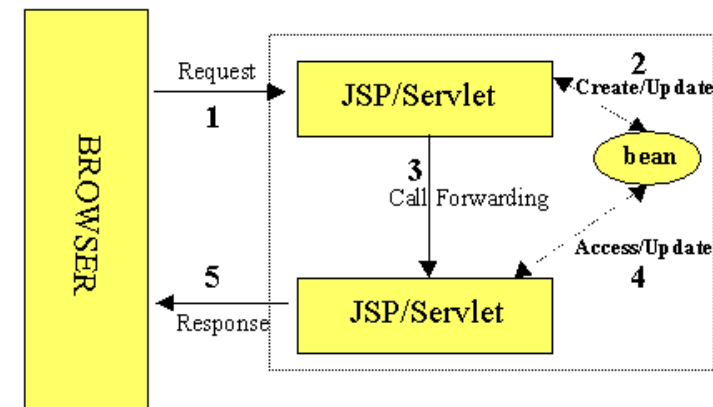
- invia dinamicamente la richiesta ad una data URL e ne include il risultato

```
<jsp:include page="shoppingCart.jsp" />
```

- **useBean**

- localizza ed istanzia (se necessario) un *javaBean* nel contesto specificato
- Il contesto può essere
 - La pagina, la richiesta, la sessione, l'applicazione

```
<jsp:useBean id="cart" scope="session" class="ShoppingCart" />
```



Elementi di scripting

- **Declaration** `<%! declaration [declaration] ... %>`

- Variabili o metodi statici usati nella pagina

```
<%! int[] v= new int[10]; %>
```

- **Expression** `<%= expression %>`

- Una espressione nel linguaggio di scripting (Java) che viene valutata al momento della richiesta e sostituita al tag

```
<p> La radice di 2 vale <%= Math.sqrt(2.0) %> </p>
```

- **Scriptlet** `<% codice %>`

- Frammenti di codice che controllano la generazione del codice HTML, valutati alla richiesta

```
<table>
    <% for (int i=0; i< v.length; i++) { %>
        <tr><td> <%= v[i] %> </td></tr>
    <% } %>
</table>
```

Tale codice diventerà parte dei metodi doGet (o doPost) della servlet che viene associata alla JSP

Elementi di scripting

- Linguaggio di script ha lo scopo di
 - interagire con oggetti java e altre servlet
 - gestire le eccezioni java
- Può utilizzare anche oggetti *impliciti* (sono 9) che non devono essere creati
 - `request`
 - `response`
 - `out`
 - `page`
 - `pageContext`
 - `session`
 - `application`
 - `config`
 - `exception`

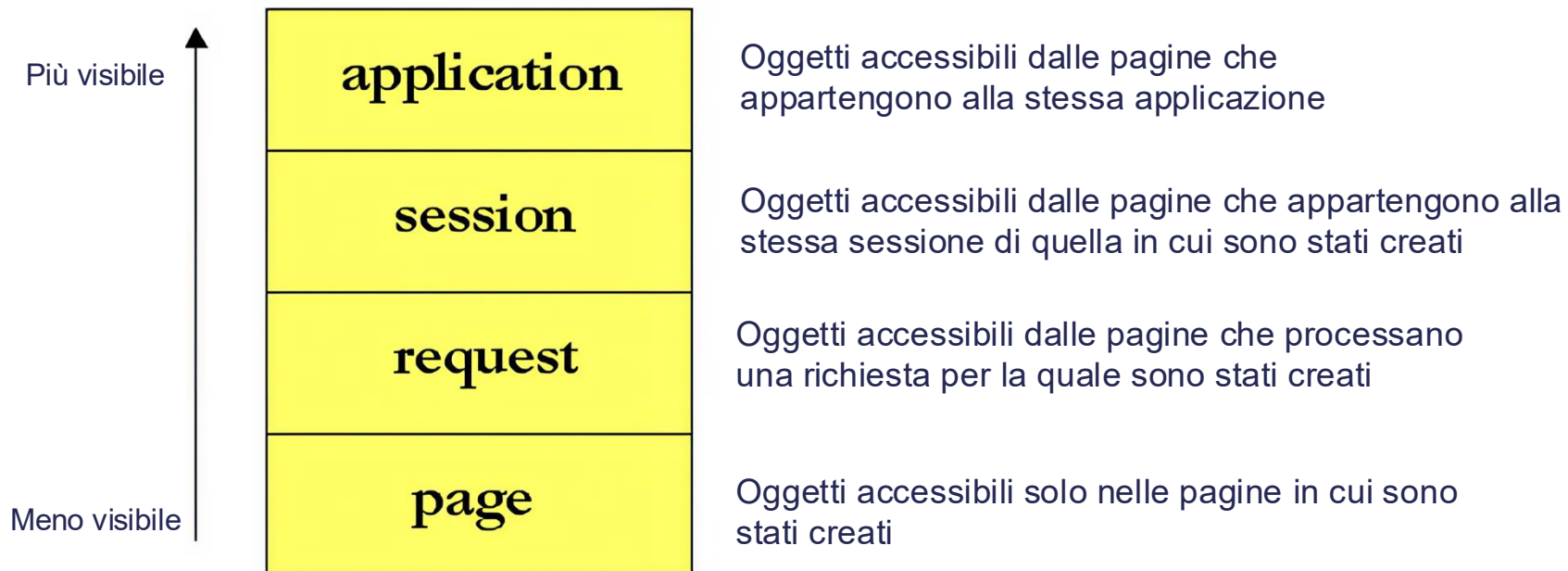
Esempio:

Grazie per la scelta di `<i> <%= request.getParameter("title") %> </i>`

Oggetti e loro “scope”

Lo **scope** (ambito) definisce **la durata e la visibilità** di un oggetto all'interno dell'applicazione web.

- I 9 **oggetti impliciti** sono inseriti dal container come variabili locali nel metodo `_jspService()`. Una JSP può usare altri oggetti creati:
 - **implicitamente**, tramite alcune direttive
 - **esplicitamente**, con le azioni (es. `<jsp:useBean>`)
 - **direttamente**, con uno scriptlet
- Ogni oggetto ha un attributo **“scope”** che ne definisce vita e visibilità.



JavaBean

- Un *javabeen* è una classe **Java semplice** (POJO – Plain Old Java Object) che segue regole precise (specifica)
 - Deve avere costruttore senza argomenti
 - Dovrebbe avere campi (property) **private**
 - I metodi di accesso ai campi (property) sono **set/get/is**

setXxx

getXxx/isXxx

con xxx = property

```
1. class Book{
2.     private String title;
3.     private boolean available;
4.     void setTitle(String t) ...;
5.     String getTitle() ...;
6.     void setAvailable(boolean b) ...;
7.     boolean isAvailable () ...;
8. }
```

La relazione tra **JSP** e **JavaBean** è centrata sull'idea di **separazione tra logica e presentazione**:

- i **JavaBean** gestiscono i **dati e la logica** dell'applicazione,
- le **JSP** si occupano di **mostrare** quei dati all'utente.

JSP e JavaBean

- Azioni per utilizzare un bean
 - Accedere ad un bean (inizializzazione)

```
<jsp:useBean id="user" class="com.esempio.Person" scope="session" />
```

```
<jsp:useBean id="user" class="com.esempio.Person" scope="session" >  
    <% user.setDate(DateFormat.getDateInstance( ).format(new Date())); //etc.. %>  
</jsp:useBean>
```

- Accedere alle proprietà

```
<jsp:getProperty name="user" property="name" />  
<jsp:setProperty name="user" property="name" value="Giovanni" />  
<jsp:setProperty name="user" property="name" value="<%= expression %>" />
```

Accesso ad un JavaBean

```
<jsp:useBean id="Attore" class="com.esempio.Actor" scope="session" type="Person"/>
```

- Lo **scope** determina la vita e visibilità del bean
 - **page**: è lo scope di default, il bean viene messo in `pageContext` ed acceduto con `getAttribute`
 - **request**: il bean viene messo in `ServletRequest` ed acceduto con `getAttribute`
 - **session** e **application**: se non esiste un bean con lo stesso id, ne viene creato uno nuovo
- Il **type** permette di assegnargli una superclasse o un'interfaccia

CounterBean.java

```
1. public class CounterBean {
2.     //declare an integer for the counter
3.     private int count;

4.     public int getCount() {
5.         //return count
6.         return count;
7.     }

8.     public void setCount(int count) {
9.         this.count = count;
10.    }

11.

12.    public void increaseCount() {
13.        //increment count
14.        count++;
15.    }
16.}
```

Gli elementi che rendono questa classe un JavaBean sono:

- **Classe pubblica.**
- **Proprietà privata (count).**
- **Metodo getter pubblico (getCount).**
- **Metodo setter pubblico (setCount).**
- **Costruttore senza argomenti** implicito (non è obbligatorio dichiararlo se non ne servono altri).

Counter.jsp

```

1.  <%@ page import="com.esempio.CounterBean"%>
2.  <jsp:useBean id="session_counter" class="com.esempio.CounterBean" scope="session" />
3.  <jsp:useBean id="app_counter" class="com.esempio.CounterBean" scope="application" />
4.  <%
5.      session_counter.increaseCount();
6.      synchronized (page) {
7.          app_counter.increaseCount();
8.      }
9.  %>
10. <h3>
11.     Number of accesses within this session:
12.     <jsp:getProperty name="session_counter" property="count" />
13. </h3>
14. <p></p>
15. <h3>
16.     Total number of accesses:
17.     <%      synchronized (page) {      %>
18.     <jsp:getProperty name="app_counter" property="count" />
19.     <%      }      %>
20. </h3>

```



Counter.jsp (senza scriptlet — logica nel JavaBean)

1. `<jsp:useBean id="session_counter" class="com.esempio.CounterBean" scope="session"/>`
2. `<jsp:useBean id="app_counter" class="com.esempio.CounterBean" scope="application"/>`
3. `<h3>Number of accesses within this session: <jsp:getProperty name="session_counter" property="andIncrement"/></h3>`
4. `<p></p>`
5. `<h3>Total number of accesses: <jsp:getProperty name="app_counter" property="andIncrement"/></h3>`

Concorrenza → responsabilità del bean

Il contatore application è condiviso tra thread: la JSP non può più usare `synchronized`. Soluzione: **spostare l'atomicità nel bean** con `AtomicInteger` e un getter `getAndIncrement()`.



Sessioni

- Accede ad un oggetto `HttpSession (session)`
- Mappa chiave-valore

HttpSession: È un oggetto lato server che memorizza dati associati a un particolare utente. Il server crea un ID di sessione univoco e lo invia al client tramite un cookie (chiamato SESSIONID). Solo questo ID viaggia tra client e server, mentre i dati rimangono sul server.

- Esempi

- Memorizzazione

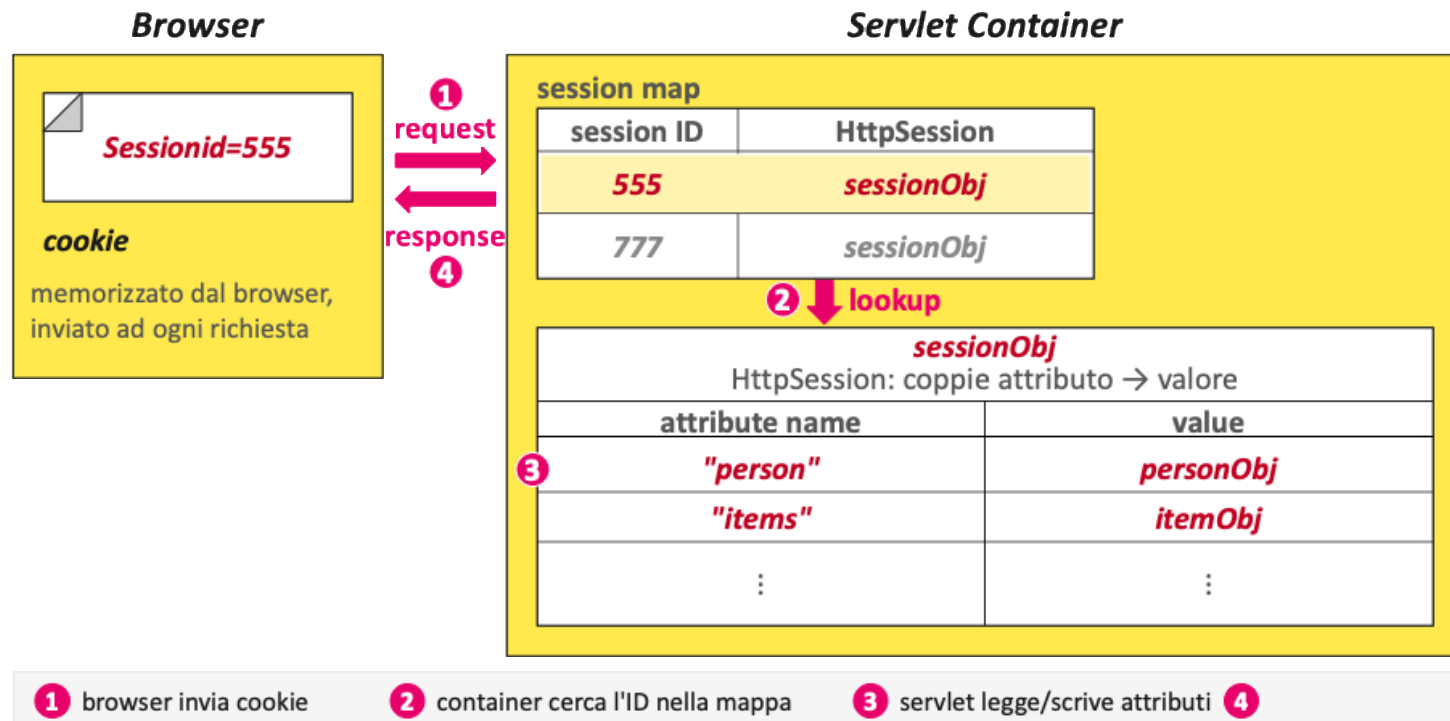
```
<% Foo foo = new Foo(); session.putValue("foo",foo); %>
```

- Recupero

```
<% Foo myFoo = (Foo) session.getValue("foo"); %>
```

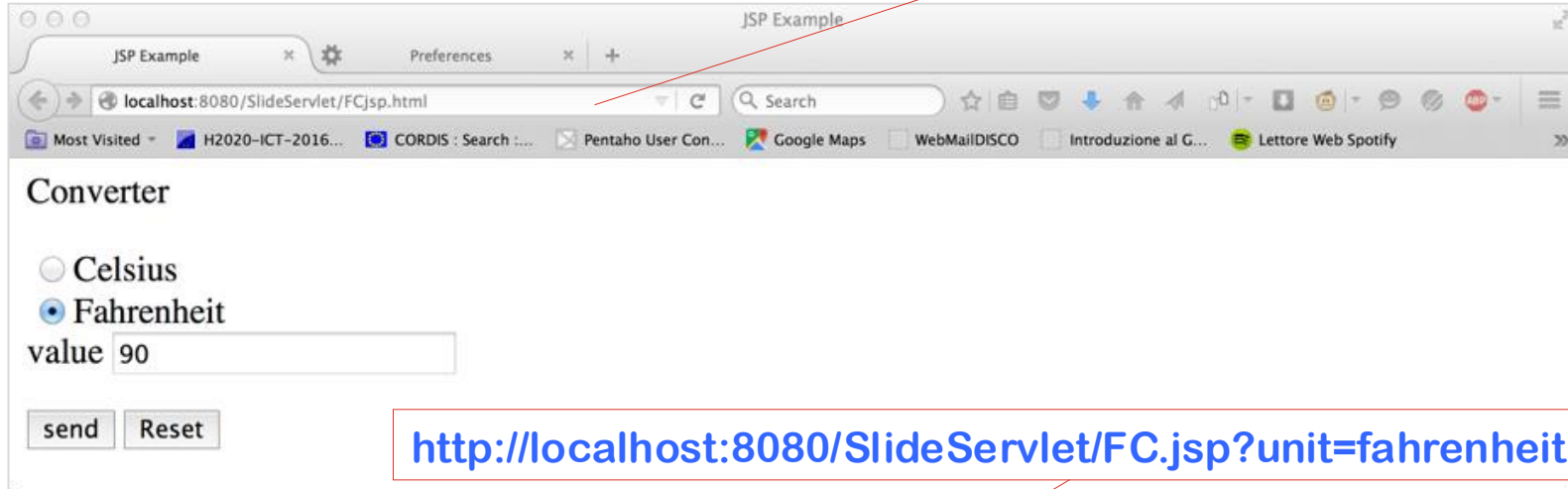
- Esclusione di una pagina dalla sessione

```
<%@ page session="false" %>
```



JSP: esempio

<http://localhost:8080/SlideServlet/FCjsp.html>



Converter

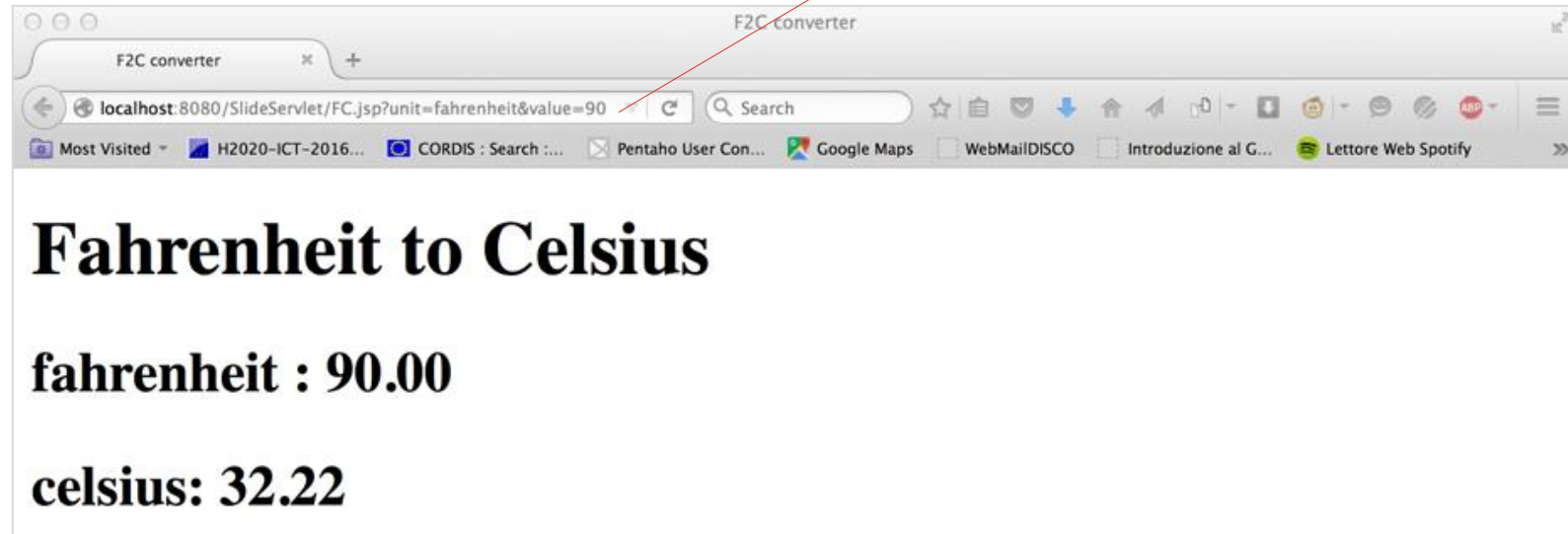
☐ Celsius

☒ Fahrenheit

value 90

send Reset

<http://localhost:8080/SlideServlet/FC.jsp?unit=fahrenheit&value=90>



F2C converter

localhost:8080/SlideServlet/FC.jsp?unit=fahrenheit&value=90

Fahrenheit to Celsius

fahrenheit : 90.00

celsius: 32.22

Un esempio JSP con Form HTML

```
1. <!DOCTYPE html>
2. <html>
3. <head>
4. <meta charset="UTF-8">
5. <title>JSP Example</title>
6. </head>
7. <body>

8. <form action="http://localhost:8080/SlideServlet/FC.jsp" method="GET">
9.     Converter <br><br>
10.    <input type="radio" name="unit" value="celsius">Celsius<br>
11.    <input type="radio" name="unit" value="fahrenheit">Fahrenheit<br>
12.    value <input type="text" name="value" ><br><br>
13.    <input type="submit" value="send">
14.    <input type="reset">
15.</form>

16.</body>
17.</html>
```

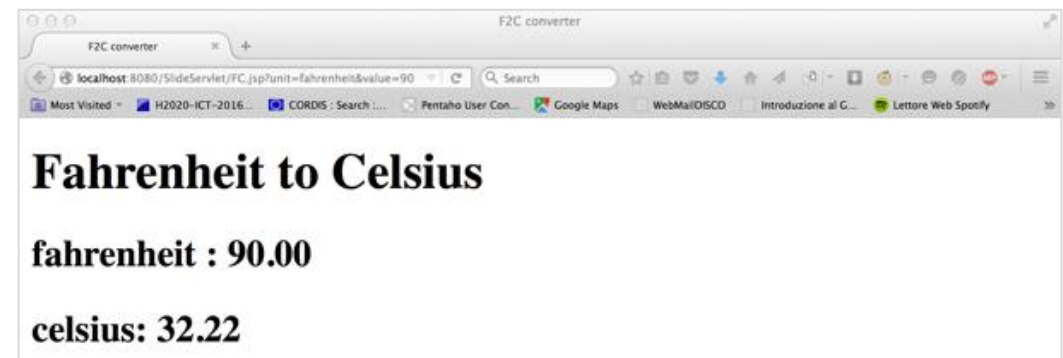


FC converter in versione JSP

```

1.  <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2.  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
3.  <html><head><meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
4.  <title>F2C converter</title></head>
5.  <body>
6.      <%@ page import = "java.text.DecimalFormat" %>
7.      <% Double result;
8.      String unit = request.getParameter("unit");
9.      Double value=Double.parseDouble(request.getParameter("value")); %>
10.     <h1> <%      if( unit.equals("fahrenheit")) { %>
11.         Fahrenheit to Celsius
12.         <% result=(value - 32) * 5 / 9;
13.         } else { %>
14.         Celsius to Fahrenheit
15.         <% result=((value * 9) / 5) + 32; } %>
16.     </h1>
17.     <% DecimalFormat twoDigits = new DecimalFormat( "#0.00" ); %>
18.     <h2> <%= unit %> : <%= twoDigits.format(value) %> </h2>
19.     <h2> <%      if( unit.equals("fahrenheit")) { %>
20.         fahrenheit:
21.         <%
22.         } else { %>
23.         celsius:
24.         <% } %>
25.         <%= twoDigits.format( result ) %>
26.     </h2>
27. </body>
28. </html>

```



FC converter rifattorizzato (JavaBean + jsp:setProperty)

ConverterBean.java

```
1 package com.esempio;
2
3 import java.text.DecimalFormat;
4
5 public class ConverterBean {
6     private String unit = "celsius";
7     private double value;
8
9     public void setUnit(String unit) {
10         this.unit = unit; }
11     public void setValue(double value) {
12         this.value = value; }
13
14     public boolean isFahrenheit() {
15         return "fahrenheit".equals(unit);
16     }
17
18     public String getResult() {
19         double result = isFahrenheit()
20             ? (value - 32) * 5 / 9
21             : value * 9 / 5 + 32;
22
23         return new DecimalFormat("#0.00").format(result);
24     } }
```

converter.jsp

```
1 <%@ page contentType="text/html; charset=UTF-8" %>
2 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
3
4 <jsp:useBean id="conv" class="com.esempio.ConverterBean" />
5 <jsp:setProperty name="conv" property="*" />
6
7 <html>
8 <head>
9     <title>Temperature converter</title>
10 </head>
11 <body>
12     <h1>
13         <c:choose>
14             <c:when test="${conv.fahrenheit}">
15                 Fahrenheit to Celsius
16             </c:when>
17             <c:otherwise>
18                 Celsius to Fahrenheit
19             </c:otherwise>
20         </c:choose>
21         :
22         <jsp:getProperty name="conv" property="result" />
23     </h1>
24 </body>
25 </html>
```

La chiave del refactoring: `property="*" —` il container JSP legge i parametri della request (unit, value) e invoca automaticamente i setter del bean (setUnit(), setValue()) convertendo i tipi. Zero `getParameter()`, zero `parseDouble()`, zero scriptlet.

Nelle slide precedenti abbiamo visto come le Servlet gestiscano la logica e le JSP la presentazione.
Ma come si organizzano questi componenti in un'architettura coerente?
Il pattern MVC definisce un modo strutturato per separare dati, presentazione e controllo.

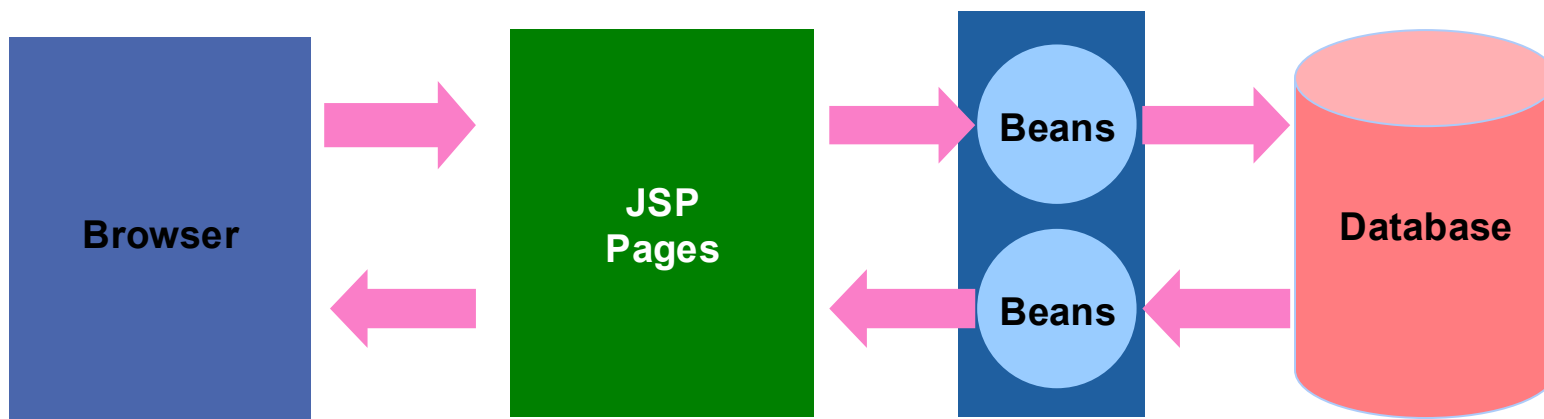
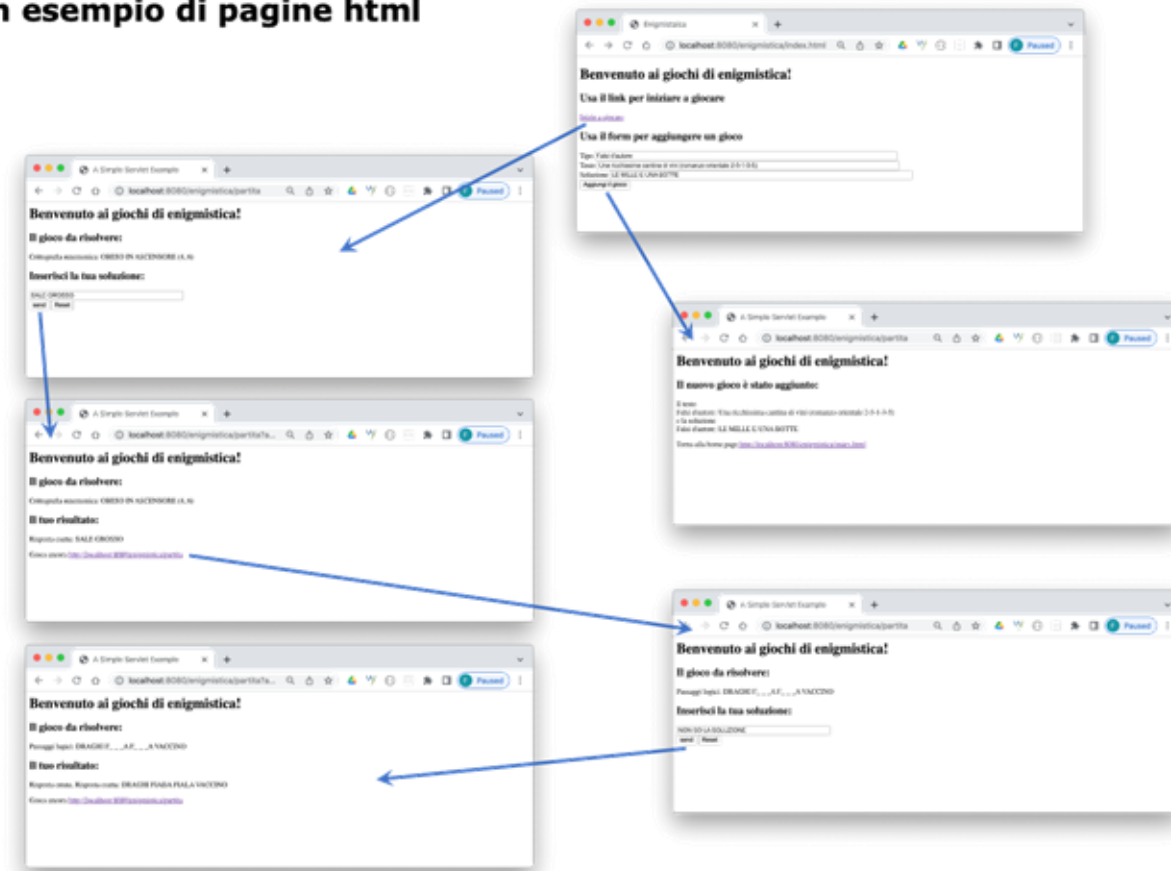
IL PATTERN MVC – MODEL VIEW CONTROL

Il pattern Model 1

Il **pattern Model 1** è una delle prime architetture usate per sviluppare applicazioni web in Java, in particolare con **JSP**.

- Scopo: separazione tra dati, e logica di business+visualizzazione
 1. Il browser invia una richiesta per la pagina JSP
 2. JSP accede a Java Bean e ne legge/aggiorna il contenuto
 3. La risposta, generata da JSP, viene inviata al browser
- La JSP è responsabile sia della gestione della richiesta sia della risposta

Un esempio di pagine html



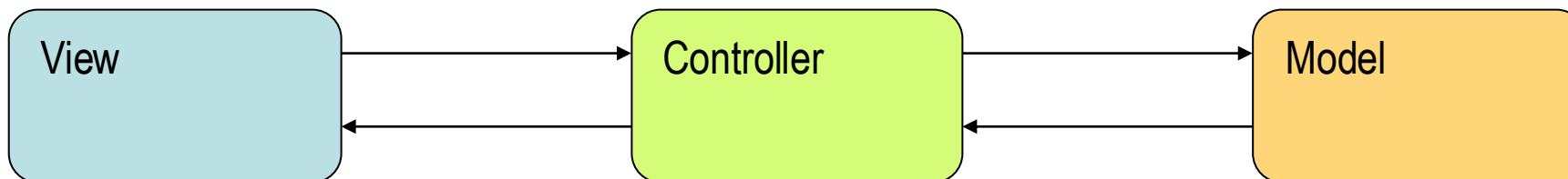
Svantaggio: il controllo della navigazione è decentralizzato, poiché ogni pagina contiene la logica per determinare la pagina successiva

Difficile da mantenere e da estendere. Inoltre per evitare che la JSP sia piena di codice java bisogna creare molti tag ad hoc

Il pattern MVC

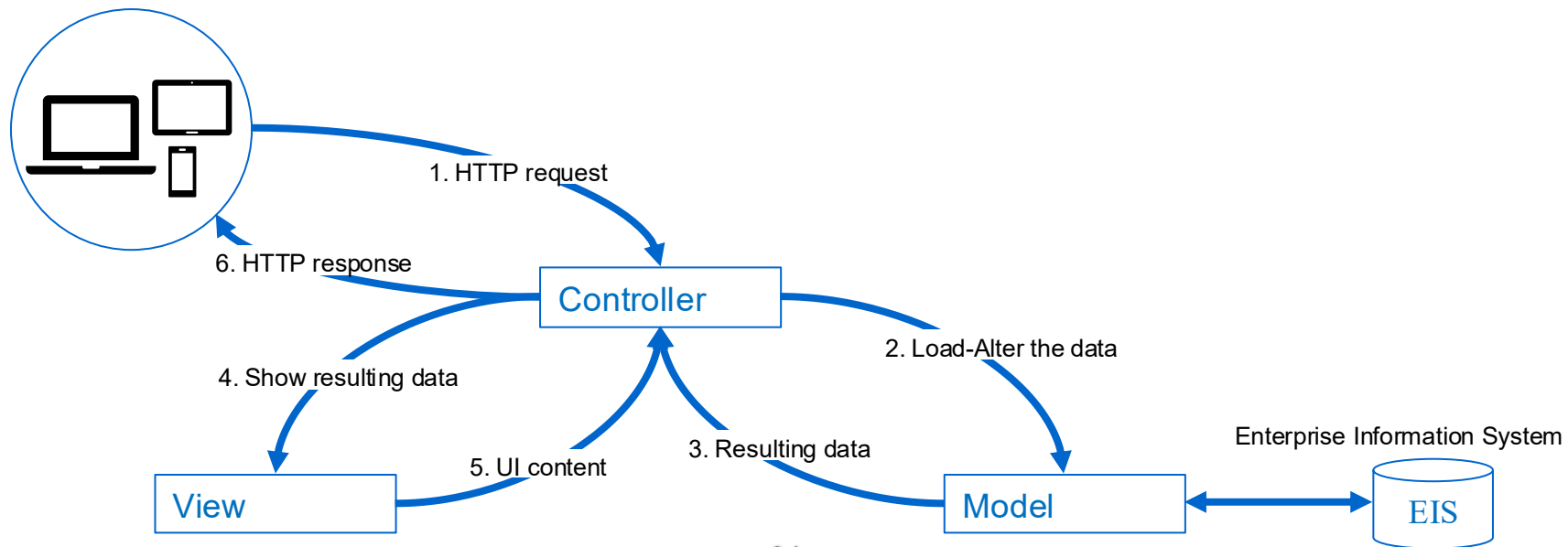
Il pattern **Model View Controller (MVC)** è pensato per separare le **responsabilità** all'interno di un'applicazione, migliorando **manutenibilità**, **modularità** e **riutilizzabilità** del codice.

- **Model** (Modello).
 - Rappresenta i dati e la logica di business. Può includere oggetti JavaBean, classi di accesso al database, validazioni, ecc.
- **View** (Vista).
 - Gestisce la presentazione dei dati all'utente. Tipicamente è costituita da JSP, HTML, CSS, ecc.
- **Controller** (Controllore).
 - Riceve e gestisce le richieste dell'utente (es. da una form).
 - Coordina il flusso tra Model e View. In Java, spesso realizzato con servlet



Il design pattern MVC

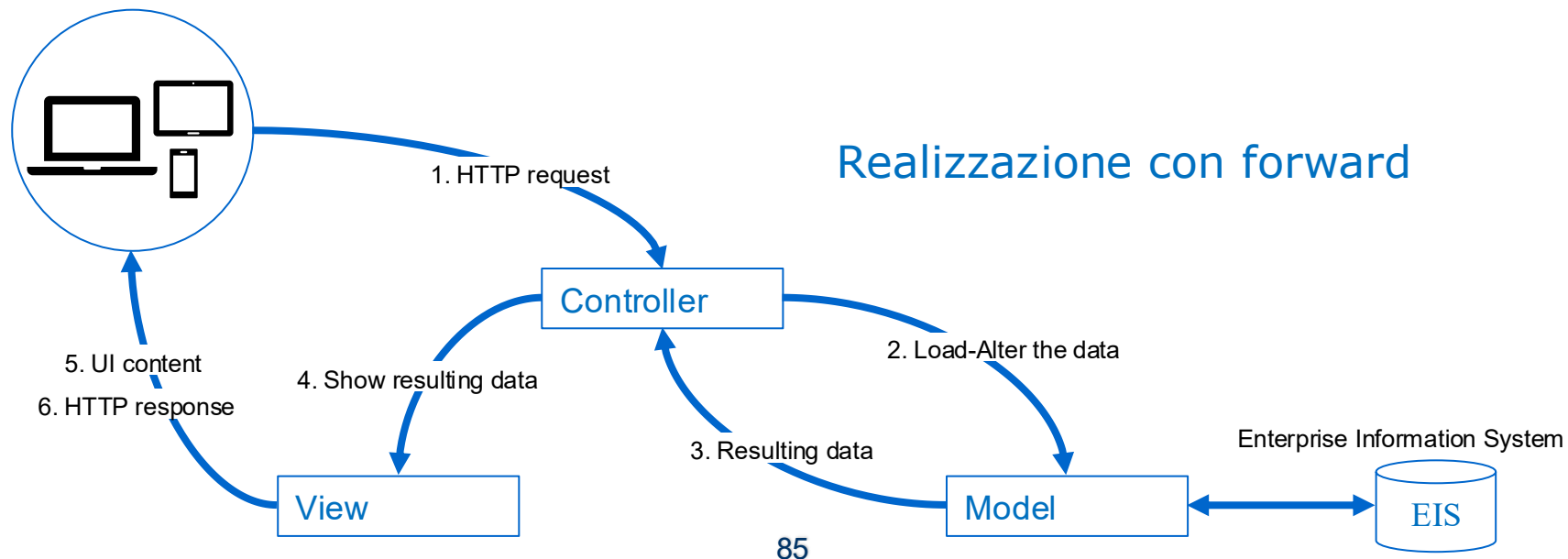
- Il Model-View-Controller (MVC) è un pattern architetturale che separa **data model**, **presentazione**, e **logica di controllo** in tre componenti distinte
- **Model**
 - I dati (gli oggetti) trattati dall'applicazione, e le operazioni su di essi
- **View**
 - La struttura dei dati restituiti al richiedente (e.g., la pagina HTML/CSS)
- **Control**
 - Definisce le azioni da eseguire a fronte di una richiesta
 - Interagisce con il Model per modificare i dati e con la View per generare la risposta



Il design pattern MVC — Realizzazione con forward

Variante forward: il Controller, dopo aver aggiornato il Model, delega alla View il rendering della risposta. Lo User Interface Content e la HTTP response partono direttamente dalla View, senza tornare al Controller.

- **Model**
 - I dati (gli oggetti) trattati dall'applicazione, e le operazioni su di essi
- **View**
 - La struttura dei dati restituiti al richiedente (e.g., la pagina HTML/CSS)
- **Control**
 - Definisce le azioni da eseguire a fronte di una richiesta
 - Interagisce con il Model per modificare i dati e con la View per generare la risposta



Vantaggi e svantaggi

■ Vantaggi:

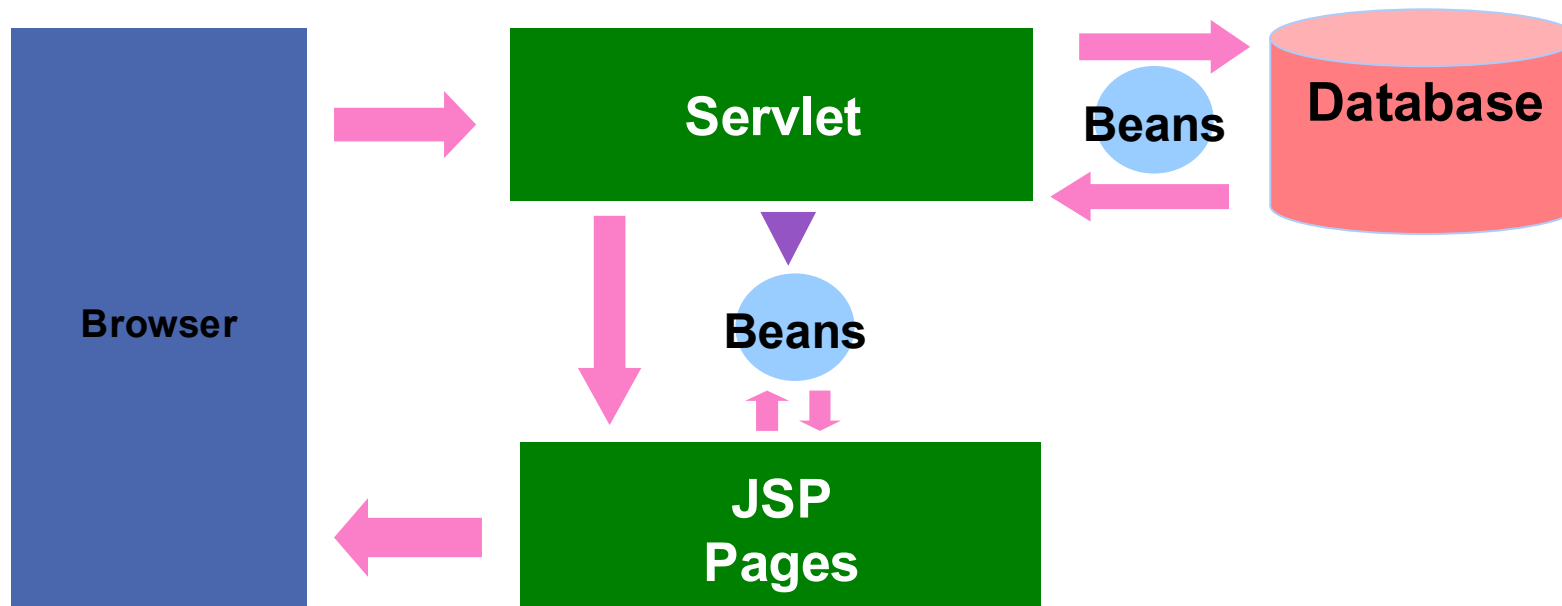
- Chiara separazione tra logica di business e logica di presentazione
 - poter cambiare la view senza modificare il control e viceversa
 - poter arricchire la view senza appesantire il codice del controller
 - poter definire e scegliere a run-time la view da usare a seconda dell'interazione, dei device utilizzati, dello stato dei dati o delle preferenze dell'utente
- Chiara separazione tra logica di business e modello dei dati
 - poter definire diversi mapping tra le azioni degli utenti sul controller e le funzioni sul model
- Ogni componente ha una responsabilità ben definita
 - poter sviluppare in parallelo
 - poter mantenere e far evolvere ogni componente in modo indipendente, quindi semplificando la gestione
- Ogni parte può essere affidata a esperti
 - poter assegnare lo sviluppo della view ad esperti di interfaccia e interazione (e.g., pagine jsp o asp)
 - uso tecnologie adatte allo sviluppo delle singole componenti

■ Svantaggi

- Aumento della complessità dovuta alla concorrenza (è un sistema distribuito)
- Inefficienza nel passaggio dei dati alla view (un elemento in più tra cliente e controller)

Il pattern Model 2 (Model View Control)

- Il **Model 2** è un'evoluzione del Model 1 e corrisponde all'applicazione del **pattern MVC (Model–View–Controller)** nelle applicazioni web Java.
- La richiesta viene inviata ad una Java Servlet (solo codice, niente HTML) che:
 - genera i dati dinamici richiesti dall'utente
 - li mette a disposizione della pagina jsp come Java Beans
- La Servlet richiama una pagina .jsp, (poco codice java) che:
 - legge i dati dai beans
 - organizza la presentazione in HTML che invia all'utente



Per concludere: Perché il modello JSP è stato superato?

Limiti architetturali

- Accoppiamento forte tra view e logica: scriptlet Java dentro l'HTML rendono il codice difficile da leggere, testare e mantenere
- Compilazione lato server ad ogni modifica: ciclo di sviluppo lento rispetto ai motori di template moderni
- Scarsa testabilità unitaria: la view non è isolabile dal container servlet

Contesto storico-evolutivo

- Nato nel 1999 per competere con ASP e PHP, in un'epoca di pagine generate interamente lato server
- L'avvento delle Single Page Application e delle API REST ha spostato la responsabilità della view sul client (browser)
- Il backend si è evoluto verso servizi stateless che espongono JSON, non HTML

Alternative moderne

- Thymeleaf (Spring): template HTML validi, separazione netta tra markup e logica
- React / Angular / Vue + REST: rendering client-side, UX reattiva, backend disaccoppiato
- Next.js / Nuxt: Server-Side Rendering ibrido che unisce i vantaggi di entrambi i mondi